

SC4002 Natural Language Processing
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1	Foundations: Tokens, Zipf, and n-Grams	1
1.1	Intuition: Why Slice Text at All?	1
1.2	Tokens, Types, and Vocabulary	1
1.3	Zipf’s Law: Few Frequent, Many Rare	3
1.4	n -Grams: Predicting the Next Token	3
1.5	Exercises	5
2	Word Representation: From One-Hot to Embeddings	6
2.1	Words Are Not Numbers: One-Hot and Its Curse	6
2.2	Distributional Hypothesis and the Embedding Matrix	7
2.3	Word2Vec: CBOW and Skip-Gram	7
2.4	GloVe: Global Vectors via Weighted Least Squares	8
2.5	Vector Arithmetic and Analogies	9
2.6	From Scratch in Python	9
2.7	Chapter Notes	10
2.8	Exercises	10
3	Text Classification: Naive Bayes to Transformers	11
3.1	The Classification Problem	11
3.2	Naive Bayes: Counting with Bayes’ Rule	12
3.3	Logistic Regression: A Linear Boundary with Probabilities	12
3.4	Transformers for Classification	13
3.5	Chapter Notes	15
3.6	Exercises	15

4	Sequence Labeling: HMM, CRF, and BiLSTM-CRF	16
4.1	Tagging as Labeling: Why Context Matters	16
4.2	Hidden Markov Models and the Viterbi Algorithm	17
4.3	Conditional Random Fields: Global Normalization	18
4.4	BiLSTM-CRF: Learning Features End-to-End	19
4.5	Chapter Notes	20
4.6	Exercises	20
5	Language Models: Smoothing and Neural Prediction	21
5.1	Predicting the Next Word: Why Counting Is Not Enough	21
5.2	Smoothing: From Laplace to Kneser–Ney	22
5.3	Neural Language Models and Perplexity	23
5.4	From Scratch in Python	25
5.5	Chapter Notes	26
5.6	Exercises	26
6	Seq2Seq: Encoder-Decoder and Attention	27
6.1	Translation as Seq2Seq: The Bottleneck Problem	27
6.2	The Encoder-Decoder Architecture	28
6.3	Attention: Letting the Decoder Look Back	28
6.4	Decoding with Beam Search and Training	29
6.5	Chapter Notes	31
6.6	Exercises	31
7	Pretraining: BERT, GPT, Fine-Tuning, and Prompting	32
7.1	Why Pretrain? The Paradigm	32
7.2	BERT: Masked Language Modeling and NSP	33
7.3	GPT: Autoregressive (Causal) Pretraining	34
7.4	Fine-Tuning, Prompting, and Efficient Adaptation	34
7.5	Chapter Notes	36
7.6	Exercises	37

8	Generation: Sampling, Alignment, and Evaluation	38
8.1	From Probabilities to Text: Why Greedy Fails	38
8.2	Sampling: Temperature, Top- k , Top- p	39
8.3	Aligning to Humans: RLHF and DPO	40
8.4	Evaluation: BLEU and ROUGE	41
8.5	Chapter Notes	43
8.6	Exercises	43

Chapter 1

Foundations: Tokens, Zipf, and n -Grams

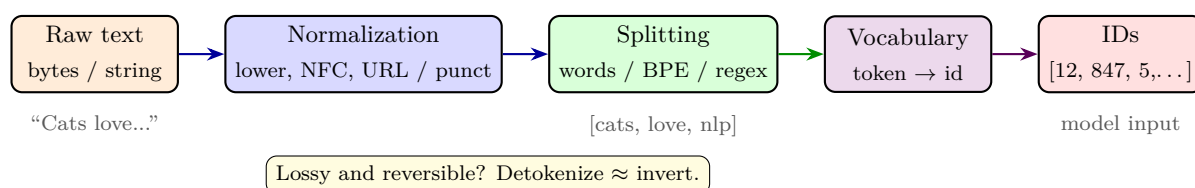
Before a machine can understand language it must slice it into pieces, count those pieces, and learn how surprising the next piece is.

1.1 Intuition: Why Slice Text at All?

Computers see only bytes; humans see words, morphemes, and meaning. Every NLP system bridges this gap with a **pipeline** that turns a raw character string into a sequence of discrete units the model can index. Imagine the sentence

"Cats love NLP!!! Visit <https://ntu.edu.sg>."

Should **Cats** equal **cats**? Should **NLP!!!** be one unit or four? Should the URL stay intact? Tokenization decides. A good tokenizer preserves meaning while keeping the vocabulary small enough to learn. Too coarse (whole sentences) and we never reuse statistics; too fine (individual bytes) and sequences become endlessly long. Modern NLP lives in the middle: words and subwords.



Normalization handles case, Unicode, and noise; splitting decides boundaries; vocabulary mapping makes everything an integer.

1.2 Tokens, Types, and Vocabulary

Definition 1.1 (Token, Type, Vocabulary). A **token** is an occurrence of a unit in running text. A **type** is the distinct equivalence class of tokens. The **vocabulary** $\mathcal{V}$ is the set of types the system recognises, with size $|\mathcal{V}| = V$. If the corpus is $[a, b, a]$, there are 3 tokens but 2 types $\{a, b\}$.

Vocabulary size drives a trade-off. Word vocabularies for English easily reach $V \approx 50\,000$ – $500\,000$ (many rare words), yet still suffer **out-of-vocabulary** (OOV) on misspellings or new names. Character vocabularies are tiny ($V \approx 256$) but force long sequences. Subwords split the difference.

Definition 1.2 (Word vs. Subword Tokenization). **Word tokenization** splits on whitespace and punctuation: **unhappiness** stays one type. **Subword tokenization** allows a word to be several types, e.g. **un## happi## ness**, so rare words are composed from frequent pieces and OOV vanishes.

The dominant subword algorithm is Byte-Pair Encoding (BPE).

Example 1.1 (BPE Intuition). Start with characters. Repeatedly merge the most frequent adjacent pair. Corpus = low low lower. Initially l o w, l o w, l o w e r. Pair lo occurs twice so merge to lo. Next low emerges, then low e etc. After a few merges the vocabulary contains low, lower, er and can encode unseen lowest as low+est.

```

1  import re
2  from collections import Counter
3
4  def whitespace_tokenize(text):
5      # naive word tokenizer: lowercase + keep words
6      text = text.lower()
7      return re.findall(r"\b\w+\b", text)
8
9  print(whitespace_tokenize("Cats love NLP!!! Visit https://ntu.edu.sg."))
10 # ['cats', 'love', 'nlp', 'visit', 'https', 'ntu', 'edu', 'sg']
11
12 def bpe_merges(corpus, num_merges=10):
13     # corpus: list of words; toy BPE counting
14     vocab = [list(w) + ['</w>'] for w in corpus]
15     for _ in range(num_merges):
16         pairs = Counter()
17         for tok in vocab:
18             for a, b in zip(tok, tok[1:]):
19                 pairs[(a, b)] += 1
20         if not pairs:
21             break
22         best = pairs.most_common(1)[0][0]
23         # merge best pair everywhere
24         new_vocab = []
25         for tok in vocab:
26             i, merged = 0, []
27             while i < len(tok):
28                 if i < len(tok)-1 and (tok[i], tok[i+1]) == best:
29                     merged.append(tok[i] + tok[i+1])
30                     i += 2
31                 else:
32                     merged.append(tok[i]); i += 1
33             new_vocab.append(merged)
34         vocab = new_vocab
35     return vocab
36
37 print(bpe_merges(["low", "low", "lower"], num_merges=5))

```

Decoding concatenates pieces and strips markers (`##` or `__`). Modern variants (WordPiece, SentencePiece, Unigram) differ only in how they score merges.

1.3 Zipf's Law: Few Frequent, Many Rare

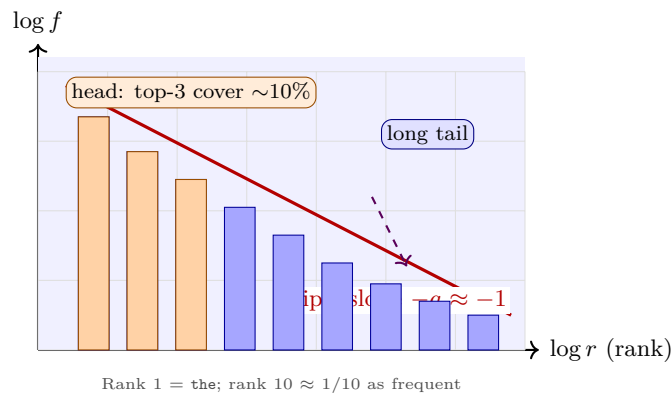
Count word frequencies in any large corpus – English novels, code, tweets. Sort types by rank $r = 1, 2, \dots, V$ (1 is most frequent). Plot frequency $f(r)$ versus r . The curve always looks the same: a steep head and a long tail.

Definition 1.3 (Zipf's Law). For rank r and constants $C > 0$, $a \approx 1$,

$$f(r) \approx \frac{C}{r^a} \quad \text{or} \quad \log f(r) \approx \log C - a \log r. \quad (1.1)$$

Thus on log–log axes Zipf's law is a straight line with slope $-a$. Often stated $f \propto 1/r$.

Intuition: language reuses a tiny core (**the**, **of**, **and**) enormously, while most words (names, technical terms) appear rarely – the **heavy tail** that makes OOV inevitable. Zipf also implies Heaps' law: vocabulary grows sublinearly $V(n) \propto n^\beta$ with corpus size n .



Why care? The tail tells us smoothing and subwords are not optional – any model that assigns zero probability to unseen n -grams will fail on real text.

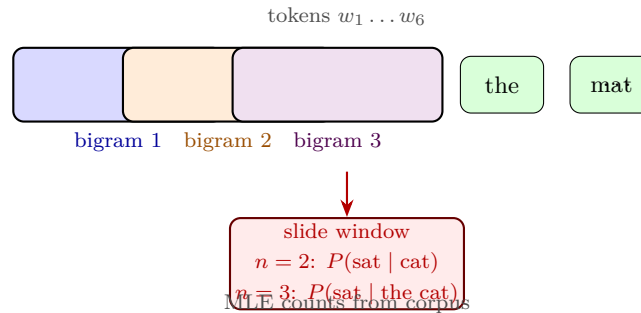
1.4 n -Grams: Predicting the Next Token

If we know the previous words, can we guess the next? An n -gram model says yes, using only the last $n - 1$ tokens.

Definition 1.4 (n -Gram and Markov Assumption). Let $w_1^m = w_1, \dots, w_m$ be a token sequence. An **n -gram** is a contiguous subsequence of length n . The model makes a $(n - 1)$ -th order Markov assumption:

$$P(w_1^m) = \prod_{i=1}^m P(w_i | w_1^{i-1}) \approx \prod_{i=1}^m P(w_i | w_{i-n+1}^{i-1}), \quad (1.2)$$

with $w_j = \langle s \rangle$ (start) for $j \leq 0$ and $w_{m+1} = \langle /s \rangle$ for sentence end. When $n = 1, 2, 3$ we call them unigram, bigram, trigram.



Maximum likelihood estimation is just counting.

Definition 1.5 (MLE for n -Grams). Let $c(\cdot)$ be corpus counts. Then

$$P_{\text{MLE}}(w_i \mid w_{i-n+1}^{i-1}) = \frac{c(w_{i-n+1}^i)}{c(w_{i-n+1}^{i-1})}. \quad (1.3)$$

In particular, for bigrams

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}, \quad (1.4)$$

and for trigrams

$$P(w_i \mid w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}. \quad (1.5)$$

Example 1.2 (Tiny Corpus). Corpus: $\langle s \rangle$ the cat sat and $\langle s \rangle$ the cat ran. Then $c(\text{the}) = 2$, $c(\text{the cat}) = 2$, $c(\text{cat sat}) = 1$. So $P(\text{cat} \mid \text{the}) = 2/2 = 1.0$, $P(\text{sat} \mid \text{cat}) = 1/2 = 0.5$. Any unseen bigram like (ran, the) gets probability 0 – the sparsity problem that motivates smoothing (Chapter 5).

```

1 from collections import Counter, defaultdict
2
3 corpus = ["<s> the cat sat </s>", "<s> the cat ran </s>"]
4 # bigram MLE
5 bigram_counts = Counter()
6 unigram_counts = Counter()
7 for sent in corpus:
8     toks = sent.split()
9     for i, w in enumerate(toks):
10         unigram_counts[w] += 1
11         if i > 0:
12             bigram_counts[(toks[i-1], w)] += 1
13
14 def p_mle(prev, word):
15     return bigram_counts[(prev, word)] / unigram_counts[prev] if unigram_counts[prev] else
16         0.0
17
18 print("P(cat|the) =", p_mle("the", "cat")) # 1.0
19 print("P(sat|cat) =", p_mle("cat", "sat")) # 0.5
20 print("P(mat|cat) =", p_mle("cat", "mat")) # 0.0 -> need smoothing

```


Chapter Summary

Text $\rightarrow$ normalization $\rightarrow$ tokenization $\rightarrow$ integer IDs. Token vs. type vs. vocabulary; word vocabularies are large and leaky, so BPE merges frequent pairs into subwords. Word frequencies follow Zipf $f \propto 1/r^a$ (straight on log-log), implying a heavy tail. n -grams model $P(w_i | n-1 \text{ history})$ by MLE ratios of counts; sparsity forces smoothing next chapter.

1.5 Exercises

Exercise 1.1. Token/type/vocabulary. Sentence: “Buffalo buffalo Buffalo buffalo.” After lowercasing and stripping punctuation you get [buffalo, buffalo, buffalo, buffalo]. How many tokens? How many types? Give $|\mathcal{V}|$. If a tokenizer keeps case, how do answers change?

Exercise 1.2. BPE trace. Corpus words: ["ab", "ac", "ab", "ad"] with characters plus $\langle w \rangle$. Show the first two BPE merges and the resulting tokenizations. Why does BPE prefer **ab** as a merged type over **ac**?

Exercise 1.3. Zipf calculation. Suppose $a = 1$, C chosen so $\sum_{r=1}^V C/r = 1$ with $V = 10\,000$. Estimate C (harmonic number $H_{10000} \approx 9.21$) and compute $f(1), f(10), f(100)$. What fraction of probability mass is in the top-10 ranks?

Exercise 1.4. Bigram MLE by hand. Corpus: three sentences $\langle s \rangle$ a b a, $\langle s \rangle$ a b, $\langle s \rangle$ b a. Compute $c(a)$, $c(b)$, $c(a, b)$, $c(b, a)$, $c(\langle s \rangle, a)$ and then $P(b | a)$, $P(a | b)$, $P(a | \langle s \rangle)$. Verify each history sums to 1.

Exercise 1.5. Sparsity and smoothing intuition. Your bigram model trained above assigns $P(\text{mat} | \text{cat}) = 0$. Explain why this zeroes the probability of any sentence containing **cat mat**, and propose two fixes (add-one smoothing and subword splitting) with one pro and one con each.

Chapter 2

Word Representation: From One-Hot to Embeddings

“You shall know a word by the company it keeps.” — J. R. Firth (1957). Before a model can reason about language, it must turn discrete words into numbers. This chapter builds that bridge: from sparse one-hot vectors to dense embeddings that capture meaning through distributional statistics.

From zero: no NLP or ML assumed. We start from “what is a vector?” and build to Word2Vec and GloVe. Every formula is preceded by a picture; vectors, matrices and softmax are defined on first use.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain why one-hot encodings suffer sparsity and orthogonality;
- (L2) state the distributional hypothesis and how it motivates embeddings;
- (L3) write the embedding lookup and the skip-gram objective with softmax and negative sampling;
- (L4) contrast CBOW versus skip-gram architectures and trade-offs;
- (L5) write the GloVe weighted least-squares objective and its weighting function.

2.1 Words Are Not Numbers: One-Hot and Its Curse

A computer stores numbers, not symbols. For vocabulary $V = \{w_1, \dots, w_{|V|}\}$ the simplest code is a *one-hot* $\mathbf{e}_i \in \{0, 1\}^{|V|}$ with a single 1 at i and 0 elsewhere. Thus $\text{cat} = [0, 1, 0, \dots]^\top$, $\text{dog} = [0, 0, 1, \dots]^\top$.

Three curses. (i) **Sparsity:** $|V| \approx 50\text{k}–500\text{k}$, almost all zeros; memory/compute scale with $|V|$. (ii) **Orthogonality:** $\mathbf{e}_i^\top \mathbf{e}_j = 0$ ($i \neq j$), so every word is equidistant (cosine 0); cat is as far from dog as from rocket .

(iii) **No generalisation:** seeing *cat* tells the model nothing about *dog*. We need dense vectors where distance mirrors meaning.

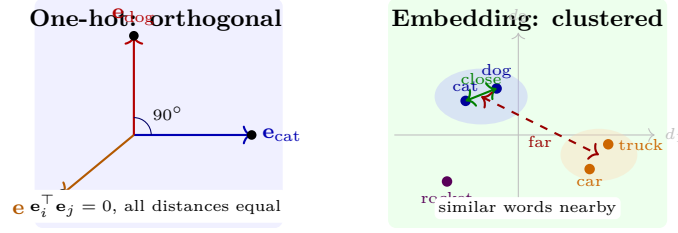


Figure 2.1: Left: one-hot axes are orthogonal (90°); every pair has distance $\sqrt{2}$. Right: dense embeddings ($d \ll |V|$, e.g. 100–300) place semantically similar words nearby; distance now carries meaning.

2.2 Distributional Hypothesis and the Embedding Matrix

Intuition. Firth’s hypothesis: words in similar contexts have similar meanings. “The *cat* sat on the mat” and “The *dog* sat on the mat” share the frame “The ____ sat on the mat”, so *cat*/*dog* should obtain nearby vectors. We learn vectors by making them *predict* context.

Formal: embedding matrix. Let $d \ll |V|$ be the embedding dimension. Collect vectors as columns of

$$\mathbf{E} \in \mathbb{R}^{d \times |V|} = [\mathbf{v}_1 \cdots \mathbf{v}_{|V|}]. \quad (2.1)$$

If $\mathbf{x}_i \in \{0, 1\}^{|V|}$ is the one-hot for w_i , lookup is

$$\mathbf{v}_i = \mathbf{E}\mathbf{x}_i \quad (\text{selects column } i). \quad (2.2)$$

Training adjusts $\mathbf{E}$ so dot products $\mathbf{v}_i^\top \mathbf{v}_j$ reflect co-occurrence. Often two matrices are kept: $\mathbf{W} \in \mathbb{R}^{|V| \times d}$ (input) and $\mathbf{W}' \in \mathbb{R}^{d \times |V|}$ (context) with vectors $\mathbf{v}_w, \mathbf{v}'_w$. Cosine $\cos(\mathbf{v}_i, \mathbf{v}_j) = \mathbf{v}_i^\top \mathbf{v}_j / (\|\mathbf{v}_i\| \|\mathbf{v}_j\|)$ measures relatedness.

2.3 Word2Vec: CBOW and Skip-Gram

Word2Vec (Mikolov et al., 2013) learns $\mathbf{E}$ by self-supervision on running text with two mirror architectures.

Skip-gram: given centre w_I , predict each context w_O in window c ($c = 2$ means two left, two right). **CBOW** does the reverse: average $2c$ context vectors to predict the centre. CBOW smooths and is faster; skip-gram treats each context separately and handles rare words better—hence more common on large corpora.

Skip-gram objective (softmax). For corpus $w_1, \dots, w_T$,

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log p(w_{t+j} \mid w_t), \quad p(w_O \mid w_I) = \frac{\exp(\mathbf{v}'_{w_O}^\top \mathbf{v}_{w_I})}{\sum_{w=1}^{|V|} \exp(\mathbf{v}'_w^\top \mathbf{v}_{w_I})}. \quad (2.3)$$

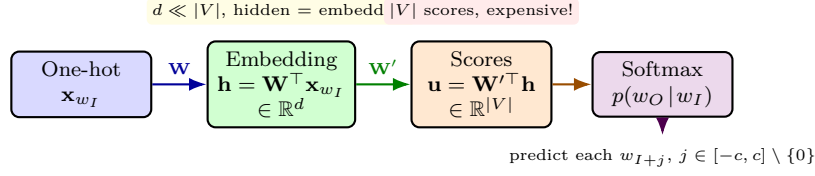


Figure 2.2: Skip-gram architecture. One-hot selects a d -dimensional hidden vector; the output layer scores all $|V|$ words. CBOW reverses the arrows: average $2c$ context embeddings to predict the centre.

The denominator sums over $|V|$ (10^5 – 10^6) per step—prohibitive.

Negative sampling. Replace softmax by k binary decisions. For true pair (w_I, w_O) and k noise words $w_i \sim P_n(w)$ (unigram^{3/4}),

$$J_{\text{NEG}} = \log \sigma(\mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I}) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n} [\log \sigma(-\mathbf{v}'_{w_i}{}^\top \mathbf{v}_{w_I})], \quad (2.4)$$

where $\sigma(x) = 1/(1 + e^{-x})$. Cost drops to $O(k)$ with $k = 5$ – 20 ; the objective pulls true pairs together and pushes noise pairs apart. CBOW uses $\bar{\mathbf{v}} = \frac{1}{2c} \sum_j \mathbf{v}_{w_{t+j}}$ in place of $\mathbf{v}_{w_I}$.

2.4 GloVe: Global Vectors via Weighted Least Squares

Word2Vec scans local windows; GloVe (Pennington et al., 2014) exploits global co-occurrence X_{ij} = times w_j occurs in the context of w_i .

Idea. Ratios of co-occurrence probabilities carry meaning. With $P(k \mid \text{ice})$ vs. $P(k \mid \text{steam})$, $k = \text{solid}$ gives ratio $\gg 1$, $k = \text{gas}$ the opposite, $k = \text{water} \approx 1$. GloVe fits

$$\mathbf{w}_i{}^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j \approx \log X_{ij}, \quad (2.5)$$

with word/context vectors $\mathbf{w}_i, \tilde{\mathbf{w}}_j \in \mathbb{R}^d$ and biases $b_i, \tilde{b}_j$.

Objective. Weighted least squares over $X_{ij} > 0$:

$$J_{\text{GLOVE}} = \sum_{i,j=1}^{|V|} f(X_{ij}) (\mathbf{w}_i{}^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij})^2, \quad (2.6)$$

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & \text{otherwise} \end{cases}, \quad \alpha = 0.75, \quad x_{\max} = 100. \quad (2.7)$$

f down-weights ultra-frequent pairs (“the”) and rare noise. The sum is over the sparse co-occurrence matrix, trainable with AdaGrad; final vector is often $\mathbf{w}_i + \tilde{\mathbf{w}}_i$.

2.5 Vector Arithmetic and Analogies

Well-trained embeddings show *linear* analogy structure, e.g. $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$: one direction encodes gender, another royalty. Analogies are tested via

$$w^* = \arg \max_w \cos(\mathbf{v}_w, \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c) \quad \text{for } a:b :: c:? . \quad (2.8)$$

Cosine underlies nearest-neighbour search and retrieval. Analogies are diagnostic, not definitional; contextual

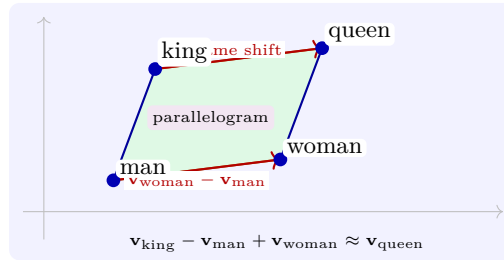


Figure 2.3: Vector analogy as a parallelogram. The gender direction $\mathbf{v}_{\text{woman}} - \mathbf{v}_{\text{man}}$ translates king to queen; closure shows the relation is approximately linear (not exact—quality depends on corpus and frequency).

embeddings (Ch. 7) refine this.

2.6 From Scratch in Python

```

1 import numpy as np
2 vocab = ["cat", "dog", "car", "truck", "rocket"]
3 V, d = len(vocab), 4
4 word_to_id = {w:i for i,w in enumerate(vocab)}
5 def one_hot(word):
6     v = np.zeros(V); v[word_to_id[word]] = 1.0; return v
7 E = np.random.randn(d, V) * 0.1 # d x V embedding matrix
8 def embed(word):
9     return E @ one_hot(word) # selects column -> R^d
10 print("cat ->", embed("cat"))
11 def sigmoid(x): return 1/(1+np.exp(-x))
12 def skipgram_neg_loss(center, outside, negatives, W, Wp):
13     """W: V x d, Wp: d x V; negative-sampling loss for one pair."""
14     v_c = W[word_to_id[center]]
15     u_o = Wp[:, word_to_id[outside]]
16     loss = -np.log(sigmoid(u_o @ v_c))
17     for neg in negatives: # push noise apart
18         u_k = Wp[:, word_to_id[neg]]
19         loss -= np.log(sigmoid(-u_k @ v_c))
20     return loss
21 # gensim-style training sketch:
22 # W, Wp = np.random.randn(V,d)*0.01, np.random.randn(d,V)*0.01
23 # for center, ctx in windows(corpus, c=2):
24 #     negs = sample_noise(k=5, dist="unigram^0.75")
25 #     loss = skipgram_neg_loss(center, ctx, negs, W, Wp)
26 #     sgd_step(W, Wp, lr=0.025) # + subsampling of frequent words

```

2.7 Chapter Notes

GloVe and Word2Vec unify count-based and prediction-based views; Levy & Goldberg (2014) show skip-gram with negative sampling implicitly factorises a shifted PMI matrix. For code, `gensim.models.Word2Vec` handles streaming/subsampling; see Mikolov et al. (2013) and Pennington et al. (2014) for original objectives.

2.8 Exercises

- E2.1 **One-hot geometry.** $|V| = 10\,000$, $d = 100$. (a) Parameters in $\mathbf{E} \in \mathbb{R}^{d \times |V|}$? (b) Show distinct one-hots have Euclidean distance $\sqrt{2}$ and cosine 0. (c) Why is nearest-neighbour meaningless under one-hot?
- E2.2 **Skip-gram cost.** $|V| = 100\,000$, $d = 300$. Estimate FLOPs for one forward pass of Eq. (2.3). With $k = 10$ negatives, what is cost of Eq. (2.4)? Explain $O(|V|) \rightarrow O(k)$.
- E2.3 **CBOW vs. skip-gram.** A corpus has many rare medical terms. Which model gives better rare-word vectors and why? What does averaging $2c$ context vectors in CBOW do to gradient variance and to rare-word sensitivity?
- E2.4 **GloVe weighting.** Sketch $f(x) = (x/100)^{0.75}$ for $0 \leq x \leq 200$. Why cap at 1 for $x \geq 100$ and use $\alpha = 0.75 < 1$? What fails if $f(x) = 1$ uniformly or $f(x) = x$?
- E2.5 **Analogy arithmetic.** With pretrained GloVe 50d (`gensim`), test $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ via cosine. Report top-3 neighbours and one *failed* analogy (morphological/polysemous). Why do static embeddings fail on polysemy?

Chapter 3

Text Classification: Naive Bayes to Transformers

“Classification is the task of choosing the right label for a piece of text. Every spam filter, sentiment analyser, and intent detector is a classifier.” — This chapter builds classifiers from counting words to attending with transformers.

From zero: no ML background assumed. We start from an email and a bag of words, then derive Naive Bayes from Bayes’ rule, logistic regression from a linear boundary, and transformer classification from self-attention and a [CLS] token. Every formula is motivated by a picture first.

Learning Outcomes

After this chapter you will be able to:

- (L1) formulate text classification with bag-of-words and decision boundaries;
- (L2) derive and apply Naive Bayes with the conditional independence assumption;
- (L3) derive the logistic sigmoid, cross-entropy loss, and its gradient;
- (L4) explain transformer classification via self-attention and the [CLS] head;
- (L5) implement and fine-tune classifiers in Python.

3.1 The Classification Problem

Intuition. You open your inbox: “WIN prize!!! Click now” vs. “Meeting at 3pm, room 101”. Without reading carefully you already guess spam or not from word clues. A classifier does exactly this—map a document x to a label $y \in \{1, \dots, K\}$. The simplest representation forgets order: a *bag-of-words* counts how often each vocabulary word occurs.

Formal setup. Fix vocabulary $V = \{w_1, \dots, w_{|V|}\}$. Document d becomes vector $\mathbf{x} \in \mathbb{N}^{|V|}$ with $x_j = \text{count}(w_j, d)$ (or TF-IDF weight). A dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ trains a function $f : \mathbb{N}^{|V|} \rightarrow \{1, \dots, K\}$. Visualise each document as a point; classification is drawing a boundary that separates colours (Fig. 3.2).

Example. Vocabulary $\{\text{win, prize, meeting, room}\}$. Email “win prize win” $\rightarrow [2, 1, 0, 0]$, label spam ($y = 1$). Email “meeting room” $\rightarrow [0, 0, 1, 1]$, label ham ($y = 0$). Even this tiny space already separates the two classes.

3.2 Naive Bayes: Counting with Bayes’ Rule

Intuition. If 90% of spam contains “prize” but only 2% of ham does, seeing “prize” is strong evidence for spam. Naive Bayes inverts the direction: from $P(\text{words} \mid \text{class})$ to $P(\text{class} \mid \text{words})$.

Formal derivation. Bayes’ rule:

$$P(y \mid \mathbf{x}) = \frac{P(y) P(\mathbf{x} \mid y)}{P(\mathbf{x})} \propto P(y) \prod_{j=1}^{|V|} P(w_j \mid y)^{x_j}, \quad (3.1)$$

where the *Naive* step assumes words are conditionally independent given y . With Laplace smoothing $\alpha > 0$:

$$P(w_j \mid y) = \frac{\text{count}(w_j, y) + \alpha}{\sum_v \text{count}(w_v, y) + \alpha|V|}, \quad \hat{y} = \arg \max_y \log P(y) + \sum_j x_j \log P(w_j \mid y). \quad (3.2)$$

Training is counting; inference is a linear score in log-space.

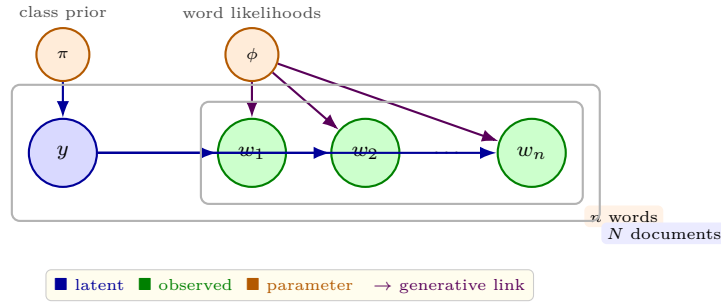


Figure 3.1: Naive Bayes generative plate diagram. For each document, sample class $y \sim \text{Cat}(\pi)$, then each word $w_i \sim \text{Cat}(\phi_y)$ independently given y . Plates replicate over words (n) and documents (N). Despite the independence assumption, NB is a strong baseline for spam and topic classification.

Fig. 3.1 shows the generative story. Learning estimates π and ϕ by counts; classification applies Eq. (3.1). Smoothing prevents zero probabilities for unseen word–class pairs.

3.3 Logistic Regression: A Linear Boundary with Probabilities

Intuition. Naive Bayes is generative (models how data is created). Logistic regression is *discriminative*: draw a straight line (hyperplane) between classes and squash its distance into a probability. Points far on one side are confidently spam.

Formal model. For binary $y \in \{0, 1\}$ with features $\mathbf{x} \in \mathbb{R}^d$:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1). \quad (3.3)$$

Decision boundary is $\mathbf{w}^\top \mathbf{x} + b = 0$ (where $P = 0.5$). Training minimises cross-entropy:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right], \quad \hat{y}^{(i)} = \sigma(\mathbf{w}^\top \mathbf{x}^{(i)} + b). \quad (3.4)$$

Gradient (derived via chain rule, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$):

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}, \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N} \sum_i (\hat{y}^{(i)} - y^{(i)}). \quad (3.5)$$

Update: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$. No closed form—optimise by gradient descent.

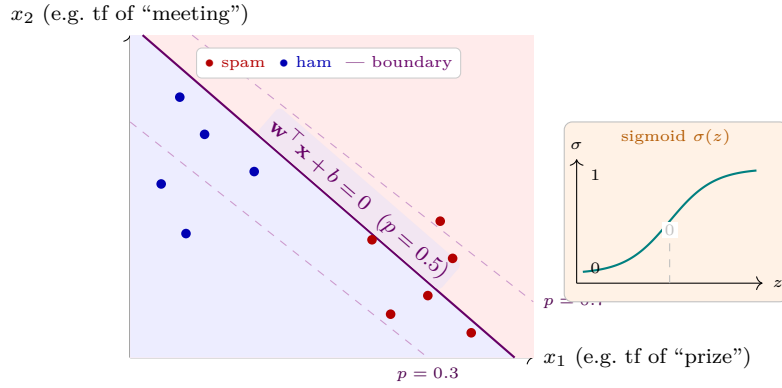


Figure 3.2: Logistic regression as a linear decision boundary. Background tint shows predicted $P(y = 1 \mid \mathbf{x})$; dashed lines are probability contours. Inset: sigmoid $\sigma(z)$ squashes the linear score $z = \mathbf{w}^\top \mathbf{x} + b$ into $(0, 1)$.

3.4 Transformers for Classification

Intuition. Bag-of-words ignores order: “dog bites man” vs. “man bites dog” look identical. Transformers read the whole sentence at once, letting every word attend to every other word, then summarise the sentence into one vector for classification.

Formal mechanism. Input tokens $[[\text{CLS}], w_1, \dots, w_n, [\text{SEP}]]$ are embedded $\mathbf{E} \in \mathbb{R}^{(n+2) \times d}$. Each layer computes self-attention:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad Q = \mathbf{E}W_Q, \quad K = \mathbf{E}W_K, \quad V = \mathbf{E}W_V, \quad (3.6)$$

with multi-head concatenation and feed-forward sublayers. After L layers, the $[\text{CLS}]$ vector $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^d$ is a contextual summary. A classification head projects it:

$$\text{logits} = \mathbf{h}_{[\text{CLS}]} W_c + \mathbf{b}_c, \quad P(y \mid \mathbf{x}) = \text{softmax}(\text{logits})_y. \quad (3.7)$$

Fine-tuning updates all weights (or just the head) by cross-entropy, using the [CLS] hidden state as the sentence vector—no manual features needed.

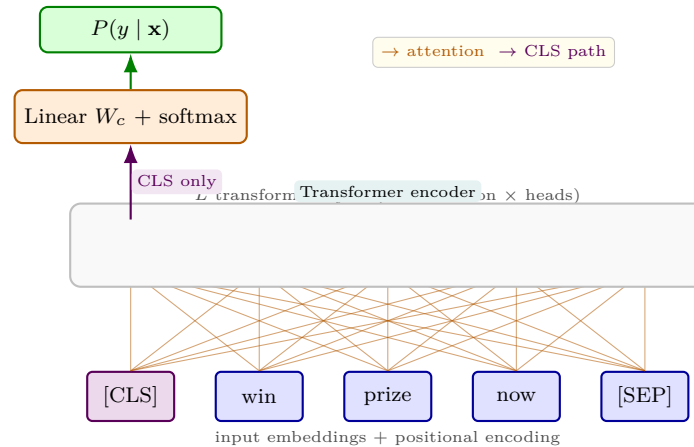


Figure 3.3: Transformer classification head. All tokens attend to all others through L layers; only $\mathbf{h}_{[\text{CLS}]}$ feeds the classifier. Fine-tuning adjusts W_Q, W_K, W_V and W_c jointly via cross-entropy on $P(y | \mathbf{x})$.

Listing 3.1: Naive Bayes from scratch (counts + Laplace) and transformer fine-tuning with Hugging Face.

```

1 import numpy as np
2 from collections import Counter, defaultdict
3
4 # --- Naive Bayes from scratch ---
5 docs = [("win prize win", 1), ("win free prize", 1), ("meeting room today", 0),
6         ("room meeting tomorrow", 0), ("free win prize", 1), ("today meeting", 0)]
7 vocab = sorted({w for d, _ in docs for w in d.split()})
8 V, alpha = len(vocab), 1.0
9 counts = {0: Counter(), 1: Counter()}
10 priors = Counter(y for _, y in docs)
11 for text, y in docs:
12     counts[y].update(text.split())
13
14 def predict(text):
15     words = text.split()
16     scores = {}
17     for y in [0, 1]:
18         logp = np.log(priors[y] / len(docs))
19         total = sum(counts[y].values())
20         for w in words:
21             pw = (counts[y][w] + alpha) / (total + alpha * V)
22             logp += np.log(pw)
23         scores[y] = logp
24     return max(scores, key=scores.get), scores
25
26 print(predict("free prize"))    # -> (1, {0: -7.1, 1: -3.8}) spam wins
27 print(predict("meeting today")) # -> (0, ...) ham wins
28
29 # --- Fine-tune BERT for classification (Hugging Face) ---
30 # pip install transformers datasets
31 from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer,
32     TrainingArguments
33 from datasets import Dataset

```

```

33
34 raw = Dataset.from_list([{"text": d, "label": y} for d, y in docs * 20])
35 tok = AutoTokenizer.from_pretrained("distilbert-base-uncased")
36 def tokenize(b): return tok(b["text"], truncation=True, padding="max_length", max_length
    =16)
37 ds = raw.map(tokenize, batched=True).train_test_split(test_size=0.2)
38
39 model = AutoModelForSequenceClassification.from_pretrained(
40     "distilbert-base-uncased", num_labels=2)
41 args = TrainingArguments(output_dir="./bert-cls", per_device_train_batch_size=8,
42     num_train_epochs=3, learning_rate=2e-5, logging_steps=5)
43 Trainer(model=model, args=args, train_dataset=ds["train"],
44     eval_dataset=ds["test"], tokenizer=tok).train()

```

3.5 Chapter Notes

Naive Bayes dates to the 1960s spam filters; despite independence being false, it remains competitive on small data. Logistic regression (Cox, 1958) is the discriminative counterpart—same linear form, different training objective. Transformers for classification were popularised by BERT [?]: the [CLS] trick turns any encoder into a classifier by fine-tuning. For deeper theory see Bishop Ch. 4 (generative vs. discriminative) and Jurafsky & Martin Ch. 4–11.

3.6 Exercises

- E3.1 **NB by hand.** With the six documents above and $\alpha = 1$, compute $P(\text{"prize"} \mid y = 1)$, $P(\text{"meeting"} \mid y = 0)$, and $P(y = 1 \mid \text{"win meeting"})$ via Eq. (3.1). Show smoothing changes the result vs. $\alpha = 0$.
- E3.2 **Decision boundary.** Let $\mathbf{w} = [2, -1]^\top$, $b = -1$. (a) Sketch $\mathbf{w}^\top \mathbf{x} + b = 0$ for $\mathbf{x} \in \mathbb{R}^2$ and label the $p > 0.5$ side. (b) Compute $\sigma(\mathbf{w}^\top \mathbf{x} + b)$ for $\mathbf{x} = [1, 1]^\top$ and $[3, 0]^\top$. (c) Explain why logistic regression cannot separate XOR points $\{(0, 0), (1, 1)\}$ vs. $\{(0, 1), (1, 0)\}$.
- E3.3 **Gradient step.** For one example $\mathbf{x} = [1, 2]^\top$, $y = 1$, current $\mathbf{w} = [0, 0]^\top$, $b = 0$, compute $\hat{y}$, the loss, and one SGD step with $\eta = 0.5$. Repeat for $y = 0$.
- E3.4 **CLS vs. pooling.** A colleague averages all $\mathbf{h}_i$ instead of using $\mathbf{h}_{[\text{CLS}]}$ for classification. (a) Give one advantage of [CLS] (hint: pre-training objective). (b) Describe a case where mean-pooling might help. (c) How does self-attention ensure $\mathbf{h}_{[\text{CLS}]}$ still sees every word?
- E3.5 **Build and compare.** Extend the NB code to (a) use TF-IDF weights instead of counts, (b) evaluate accuracy on a held-out split, and (c) compare against `sklearn.linear_model.LogisticRegression` and the fine-tuned DistilBERT. Report which wins on 6 vs. 600 documents and explain why.

Chapter 4

Sequence Labeling: HMM, CRF, and BiLSTM-CRF

“A word in isolation is ambiguous; a word in context is a decision.” — This chapter turns tagging into a structured prediction problem where labels depend on each other.

From zero: no sequence background assumed. We start from the everyday idea of *labeling each word* (is it a noun? a person name?) and ask why independent classifiers fail. Then we formalize dependencies via HMMs, fix their bias with CRFs, and finally learn features with BiLSTM-CRF. Every formula is motivated by a picture first.

Learning Outcomes

After this chapter you will be able to:

- (L1) frame POS tagging and NER as sequence labeling and explain why label dependencies matter;
- (L2) define HMM parameters (transition, emission) and decode with the Viterbi algorithm;
- (L3) contrast locally normalized MEMMs with globally normalized CRFs and write the CRF objective;
- (L4) describe the BiLSTM-CRF architecture and why it combines representation learning with structured decoding;
- (L5) implement Viterbi decoding and apply it to a tagging example.

4.1 Tagging as Labeling: Why Context Matters

Intuition. Read “*Time flies like an arrow.*” Is *flies* a verb or noun? In “*fruit flies*” it is a noun. The same word gets different tags depending on neighbours. Sequence labeling assigns a label y_t to each token x_t in $\mathbf{x} = (x_1, \dots, x_T)$, producing $\mathbf{y} = (y_1, \dots, y_T)$. Two canonical tasks: *POS tagging* (labels like NOUN, VERB,

ADJ) and *NER* (labels like B-PER, I-PER, B-LOC, O with BIO scheme).

Why not classify each position independently? An independent classifier predicts y_t from x_t alone, ignoring that tags have grammar: determiners precede nouns, I-PER cannot follow O without B-PER. Dependencies are constraints. If we predict $\hat{y}_t = \arg \max_y P(y | x_t)$ per position, we may output invalid sequences (e.g., B-PER followed by I-LOC). Structured models score *entire sequences* jointly, trading local confidence for global coherence.

Example. Sentence: “*Alice visited Paris*” $\rightarrow$ NER tags B-PER, O, B-LOC. Knowing $y_1 = \text{B-PER}$ makes $y_2 = \text{O}$ more likely than I-PER, because a person name rarely continues with a verb. The model must learn transition preferences $P(y_t | y_{t-1})$ alongside word evidence.

4.2 Hidden Markov Models and the Viterbi Algorithm

Intuition. Imagine hidden weather (Sunny/Rainy) generating what you observe (walk/shop/clean). You see the actions, not the weather. An HMM assumes hidden states y_t evolve as a Markov chain and each emits an observation x_t .

Formal definition. An HMM defines K hidden states and vocabulary V :

$$A_{ij} = P(y_t = j | y_{t-1} = i) \quad \text{transition, } K \times K \quad (4.1)$$

$$B_j(w) = P(x_t = w | y_t = j) \quad \text{emission, } K \times V \quad (4.2)$$

$$\pi_j = P(y_1 = j) \quad \text{initial distribution.}$$

The joint probability of a state–observation pair factorizes:

$$P(\mathbf{y}, \mathbf{x}) = \pi_{y_1} B_{y_1}(x_1) \prod_{t=2}^T A_{y_{t-1}, y_t} B_{y_t}(x_t). \quad (4.3)$$

Learning estimates A, B, π by counting (MLE) or EM (Baum-Welch) when states are unobserved.

Decoding: Viterbi. At test time we want $\mathbf{y}^* = \arg \max_{\mathbf{y}} P(\mathbf{y} | \mathbf{x}) \propto P(\mathbf{y}, \mathbf{x})$. The trellis has $K \times T$ nodes; Viterbi is dynamic programming:

$$\delta_t(j) = \max_{y_{1:t-1}} P(y_{1:t-1}, y_t = j, x_{1:t}), \quad \delta_t(j) = \max_i [\delta_{t-1}(i) A_{ij}] \cdot B_j(x_t), \quad (4.4)$$

with backpointers $\psi_t(j) = \arg \max_i \delta_{t-1}(i) A_{ij}$. Complexity $O(TK^2)$, linear in length. Fig. 4.1 visualizes the trellis.

HMMs are generative: they model $P(\mathbf{x}, \mathbf{y})$. This wastes capacity on $P(\mathbf{x})$ and forces strong independence assumptions (emission depends only on current tag). Discriminative successors avoid this.

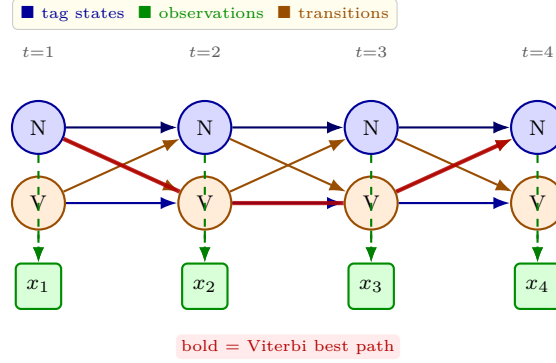


Figure 4.1: HMM trellis for POS tagging ($K = 2$: N/V). Circles are hidden tags, squares are words x_t , dashed arrows are emissions $B_j(x_t)$, solid arrows are transitions A_{ij} . Viterbi finds the maximum-weight path left-to-right.

4.3 Conditional Random Fields: Global Normalization

Intuition. A Maximum Entropy Markov Model normalizes locally at each position: $\sum_{y_t} P(y_t | y_{t-1}, \mathbf{x}) = 1$. If a state has only one outgoing transition, it always passes probability 1 regardless of the observation—*label bias*. Picture a narrow corridor: all flow is forced through it even if the wall evidence says stop.

Formal definition. A linear-chain CRF models $P(\mathbf{y} | \mathbf{x})$ directly with global normalization:

$$P(\mathbf{y} | \mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp\left(\sum_{t=1}^T \sum_{k=1}^K \lambda_k f_k(y_{t-1}, y_t, \mathbf{x}, t)\right), \quad (4.5)$$

where f_k are feature functions (e.g., $f = \mathbf{1}\{y_t = \text{B-PER}, x_t = \text{“Alice”}\}$ or $f = \mathbf{1}\{y_{t-1} = \text{B-PER}, y_t = \text{I-PER}\}$), λ_k are learned weights, and partition function

$$Z(\mathbf{x}) = \sum_{\mathbf{y}'} \exp\left(\sum_{t,k} \lambda_k f_k(y'_{t-1}, y'_t, \mathbf{x}, t)\right) \quad (4.6)$$

sums over *all* K^T label sequences—normalized once globally, not per step. This eliminates label bias: a path with weak observation support gets low global score even if local transitions are forced.

Training and inference. Log-likelihood $\sum_i \log P(\mathbf{y}^{(i)} | \mathbf{x}^{(i)})$ is concave; gradients require $Z(\mathbf{x})$ and marginals computed by forward-backward (same $O(TK^2)$). Decoding again uses Viterbi on the score $\sum_{t,k} \lambda_k f_k$. Fig. 4.2 shows the factor graph: factors couple adjacent labels and the whole observation.

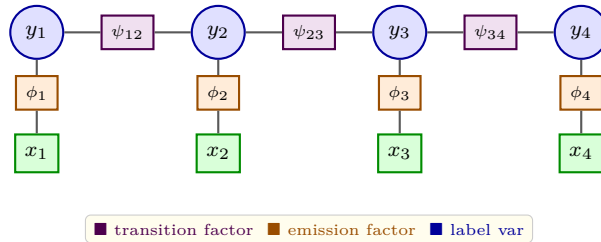


Figure 4.2: Linear-chain CRF factor graph. Variables y_t (blue) connect via pairwise factors $\psi(y_{t-1}, y_t, \mathbf{x})$ (violet) and unary factors $\phi(y_t, \mathbf{x})$ (orange) to observations x_t (green). Global $Z(\mathbf{x})$ couples all factors.

4.4 BiLSTM-CRF: Learning Features End-to-End

Intuition. CRFs need hand-crafted features f_k . A BiLSTM learns them: read the sentence left-to-right and right-to-left, so each position sees full context, then let the CRF enforce tag grammar on top.

Architecture. For token x_t , embed $\mathbf{e}_t$, run forward LSTM $\vec{\mathbf{h}}_t$ and backward LSTM $\overleftarrow{\mathbf{h}}_t$, concatenate $\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$. A linear layer maps $\mathbf{h}_t$ to emission scores $\mathbf{p}_t \in \mathbb{R}^K$ (one per tag). A learned transition matrix $\mathbf{T} \in \mathbb{R}^{K \times K}$ scores T_{y_{t-1}, y_t} . Total sequence score:

$$s(\mathbf{x}, \mathbf{y}) = \sum_{t=1}^T (p_{t, y_t} + T_{y_{t-1}, y_t}), \quad P(\mathbf{y} | \mathbf{x}) = \frac{\exp s(\mathbf{x}, \mathbf{y})}{\sum_{\mathbf{y}'} \exp s(\mathbf{x}, \mathbf{y}')}. \quad (4.7)$$

Training maximizes log-likelihood via forward-backward; decoding is Viterbi over $(\mathbf{p}, \mathbf{T})$. Fig. 4.3 sketches the stack.

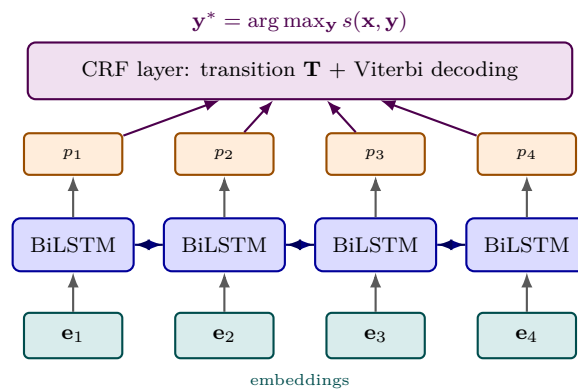


Figure 4.3: BiLSTM-CRF architecture. Embeddings $\rightarrow$ BiLSTM (forward solid, backward dashed) $\rightarrow$ per-tag emission scores $\mathbf{p}_t \rightarrow$ CRF with transition matrix $\mathbf{T}$ and global Viterbi decoding. Colours separate representation (teal/blue) from structured inference (orange/violet).

BiLSTM-CRF became the pre-Transformer workhorse for NER: BiLSTM captures long-range lexical context, CRF prevents illegal BIO transitions (e.g., I-PER without B-PER).

Listing 4.1: Viterbi decoding for HMM/CRF: works for both $A \cdot B$ and $(\mathbf{p}, \mathbf{T})$ scores.

```

1 import numpy as np
2
3 def viterbi(emission, trans, init):
4     """emission: T x K, trans: K x K, init: K. Returns best path."""
5     T, K = emission.shape
6     dp = np.zeros((T, K))          # best log-prob to reach (t,j)
7     back = np.zeros((T, K), dtype=int)
8     dp[0] = np.log(init + 1e-12) + np.log(emission[0] + 1e-12)
9     for t in range(1, T):
10         for j in range(K):
11             scores = dp[t-1] + np.log(trans[:, j] + 1e-12)
12             back[t, j] = np.argmax(scores)
13             dp[t, j] = scores[back[t, j]] + np.log(emission[t, j] + 1e-12)
14     # backtrack
15     path = [int(np.argmax(dp[T-1]))]
16     for t in range(T-1, 0, -1):

```

```

17         path.append(int(back[t, path[-1]]))
18     return list(reversed(path)), float(np.max(dp[T-1]))
19
20 # Toy POS: tags [N, V], sentence "flies like"
21 emit = np.array([[0.1, 0.8], [0.4, 0.6]]) # P(word|tag)
22 trans = np.array([[0.7, 0.3], [0.4, 0.6]]) # P(tag_t|tag_{t-1})
23 init = np.array([0.6, 0.4])
24 print(viterbi(emit, trans, init)) # e.g. [1, 1] = V V

```

4.5 Chapter Notes

HMMs were introduced by Baum and Petrie (1966); Viterbi (1967) gave the decoding algorithm. Lafferty, McCallum and Pereira (2001) defined CRFs to fix label bias in MEMMs. Huang, Xu and Yu (2015) popularized BiLSTM-CRF for sequence tagging, now often replaced by Transformer-CRF. For BIOES details see CoNLL-2003 NER shared task.

4.6 Exercises

- E4.1 **Independent vs. structured.** Give a 3-word sentence and a tag set of size 2 where per-position argmax yields a different sequence than joint argmax. Compute both predictions with concrete $P(y_t | x_t)$ and A_{ij} to show the gap.
- E4.2 **Viterbi by hand.** HMM with tags $\{N, V\}$, $A = [(0.8, 0.2), (0.3, 0.7)]$, $B_N(\text{"flies"}) = 0.1$, $B_V(\text{"flies"}) = 0.6$, $\pi = (0.5, 0.5)$. For $\mathbf{x} = (\text{"flies"}, \text{"flies"})$, fill $\delta_t(j)$ and $\psi_t(j)$ tables and backtrack the best path. What is $P(\mathbf{y}^*, \mathbf{x})$?
- E4.3 **Label bias.** Construct a 2-state MEMM where state s_1 has one outgoing edge and s_2 has two. Show that $P(y_t | y_{t-1} = s_1, \mathbf{x}) = 1$ regardless of $\mathbf{x}$, while a CRF with the same features can assign low probability to the forced path. Explain why global $Z(\mathbf{x})$ fixes this.
- E4.4 **CRF feature design.** Write 5 feature functions $f_k(y_{t-1}, y_t, \mathbf{x}, t)$ useful for NER on “Bank of America”: include one transition, one word-identity, one capitalization, one gazetteer, and one suffix feature. Give each a weight sign and justify.
- E4.5 **BiLSTM-CRF extension.** Explain why emission scores $\mathbf{p}_t$ come from $\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$ rather than $\vec{\mathbf{h}}_t$ alone. Sketch how to replace BiLSTM with a Transformer encoder while keeping the CRF layer unchanged, and state which equations stay the same.

Chapter 5

Language Models: Smoothing and Neural Prediction

“A language model is an imagination trained on counting: given what came before, what comes next?” — This chapter turns that question from raw counts into smoothed probabilities and then into neural networks.

From zero: no LM background assumed. We start from the everyday idea of autocompleting a sentence, ask why counting n -grams always leaves gaps, and fix those gaps by smoothing. Then we replace tables of counts with a network that predicts the next word and measure it with perplexity. Every formula is motivated by a picture first.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain the chain rule and the Markov assumption for n -gram models;
- (L2) write the MLE and explain why sparsity forces smoothing;
- (L3) compare Laplace, Good–Turing, and interpolated Kneser–Ney smoothing;
- (L4) describe feedforward and RNN neural LMs (softmax + cross-entropy);
- (L5) define perplexity as 2^H and use it to compare models.

5.1 Predicting the Next Word: Why Counting Is Not Enough

Intuition. Type “the cat sat on the” into your phone. It suggests “mat” before you finish. A **language model** (LM) assigns a probability $P(w_1^m)$ to any word sequence, or equivalently $P(w_i \mid w_1^{i-1})$ to the next word

given history. By the chain rule

$$P(w_1^m) = \prod_{i=1}^m P(w_i | w_1^{i-1}), \quad w_i \in \mathcal{V}. \quad (5.1)$$

An n -gram makes a $(n-1)$ -th order Markov assumption: keep only the last $n-1$ words, e.g. $P(w_i | w_{i-2}, w_{i-1})$ for $n=3$. The table is finite, so it can be learned by counting.

Formal: MLE. Let $c(\cdot)$ be corpus counts. The maximum-likelihood estimate is the count ratio

$$P_{\text{MLE}}(w_i | w_{i-n+1}^{i-1}) = \frac{c(w_{i-n+1}^i)}{c(w_{i-n+1}^{i-1})}, \quad P_{\text{MLE}}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}. \quad (5.2)$$

Example: corpus = “the cat sat”, “the cat ran”. Then $P_{\text{MLE}}(\text{sat} | \text{cat}) = 1/2$, $P_{\text{MLE}}(\text{mat} | \text{cat}) = 0/2 = 0$.

The sparsity problem. With $|\mathcal{V}| = 50\,000$, there are 2.5B possible bigrams; any corpus sees a tiny fraction. An MLE of 0 zeroes the whole sentence probability via Eq. (5.1): one unseen n -gram kills the product. Worse, Zipf’s long tail guarantees most n -grams are unseen at test time. **Smoothing** redistributes a little probability from seen to unseen events so no n -gram ever gets exactly 0.

5.2 Smoothing: From Laplace to Kneser–Ney

Laplace (add- α) smoothing. The simplest fix: pretend every n -gram occurred α times more. For bigrams with vocabulary size V ,

$$P_{\text{Lap}}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) + \alpha}{c(w_{i-1}) + \alpha V}, \quad \alpha = 1 \text{ is add-one.} \quad (5.3)$$

Intuition: we smooth by pulling every estimate toward the uniform $1/V$. Cheap but blunt—it steals too much from frequent n -grams when V is large. With $V = 50\,000$, add-one adds 50 000 phantom counts per history.

Good–Turing: let the counts tell us. Count how many n -grams occurred r times: $N_r = |\{n\text{-gram} : c = r\}|$. Good–Turing re-estimates count r as

$$r^* = (r+1) \frac{N_{r+1}}{N_r}, \quad P_{\text{GT}}(w_{i-n+1}^i) = \frac{r^*}{N}, \quad N = \sum_r r N_r. \quad (5.4)$$

Events seen once ($r=1$) are discounted using the number seen twice (N_2); unseen events ($r=0$) get mass N_1/N in total—the proportion of singletons. Intuition: the frequency of rare events predicts the chance of unseen ones. Good–Turing is much better for unigrams but not conditional $P(w_i | \text{history})$ directly.

Kneser–Ney: the right backoff. Consider “San Francisco is ...”. Bigram “San Francisco” is frequent, but “Francisco” occurs almost only after “San”. If we back off to unigram $P(\text{Francisco})$, it looks common—wrong. Kneser–Ney uses **continuation probability**: how many *different* contexts does a word follow?

$$P_{\text{cont}}(w) = \frac{|\{v : c(v, w) > 0\}|}{|\{(u, v) : c(u, v) > 0\}|}. \quad (5.5)$$

“Francisco” has continuation $1/V$ (only “San” precedes it), “the” has high continuation. Interpolated Kneser–Ney for bigrams:

$$P_{\text{KN}}(w_i | w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - D, 0)}{c(w_{i-1})} + \lambda(w_{i-1}) P_{\text{cont}}(w_i), \quad (5.6)$$

where $0 < D < 1$ is a discount (typically 0.75) and

$$\lambda(w_{i-1}) = \frac{D}{c(w_{i-1})} |\{w : c(w_{i-1}, w) > 0\}| \quad (5.7)$$

is the leftover mass given to the backoff. For trigrams it recurses: $P_{\text{KN}}(w_i | w_{i-2}, w_{i-1})$ backs off to bigrams, which back off to P_{cont} . This is the n -gram gold standard before neural models.

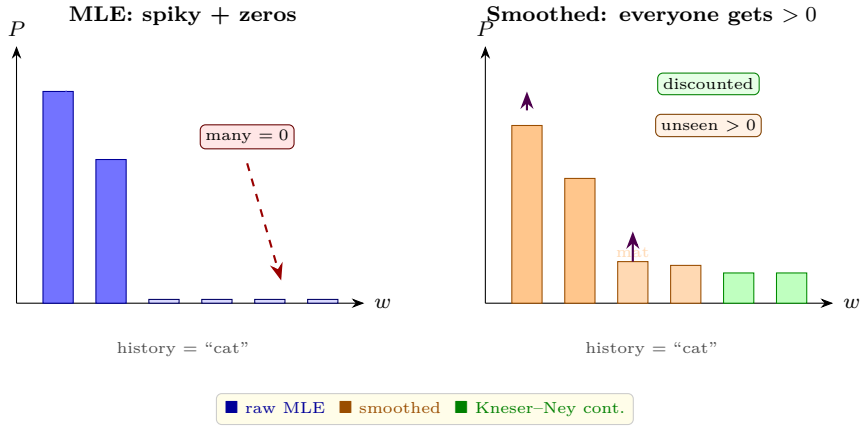


Figure 5.1: Left: MLE leaves most words at exactly zero (red), so any sentence with an unseen continuation gets probability 0. Right: smoothing discounts seen bars (violet arrow down) and lifts unseen bars above zero (violet arrow up). Kneser–Ney further reweights by continuation counts (green).

5.3 Neural Language Models and Perplexity

Feedforward NNLM (Bengio 2003). Instead of a table $P(w_i | w_{i-n+1}^{i-1})$, learn a network. Fix window n (e.g. $n = 3$). Map each context word to an embedding $\mathbf{e} = \mathbf{E}\mathbf{x} \in \mathbb{R}^d$, concatenate, and pass through a hidden layer:

$$\mathbf{h} = \tanh(\mathbf{W}[\mathbf{e}_{i-n+1}; \dots; \mathbf{e}_{i-1}] + \mathbf{b}), \quad \mathbf{z} = \mathbf{U}\mathbf{h} + \mathbf{c} \in \mathbb{R}^V, \quad P(w_i = v | \text{ctx}) = \text{softmax}(\mathbf{z})_v = \frac{e^{z_v}}{\sum_{k=1}^V e^{z_k}}. \quad (5.8)$$

Training minimizes cross-entropy (negative log-likelihood) over the corpus of length T :

$$\mathcal{L} = -\frac{1}{T} \sum_{i=1}^T \log P(w_i | w_{i-n+1}^{i-1}), \quad \nabla_{\mathbf{z}} \mathcal{L} = \mathbf{p} - \mathbf{y}_{\text{one-hot}}. \quad (5.9)$$

Similar words get similar embeddings, so “cat sat” and “dog sat” share statistics—a cure for sparsity that no count model achieves. The price is a fixed window.

RNN language model. Drop the window. An RNN keeps a hidden state that summarizes *all* prior words:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{e}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h), \quad \mathbf{z}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \quad P(w_{t+1} | w_1^t) = \text{softmax}(\mathbf{z}_t). \quad (5.10)$$

One matrix $\mathbf{W}_{hh}$ is reused every step (weight tying through time). Training via backpropagation through time (BPTT) unfolds the recurrence. In practice we use LSTM/GRU gates to keep gradients alive. RNNs have unbounded context in principle; Transformers later make that context fully parallel.

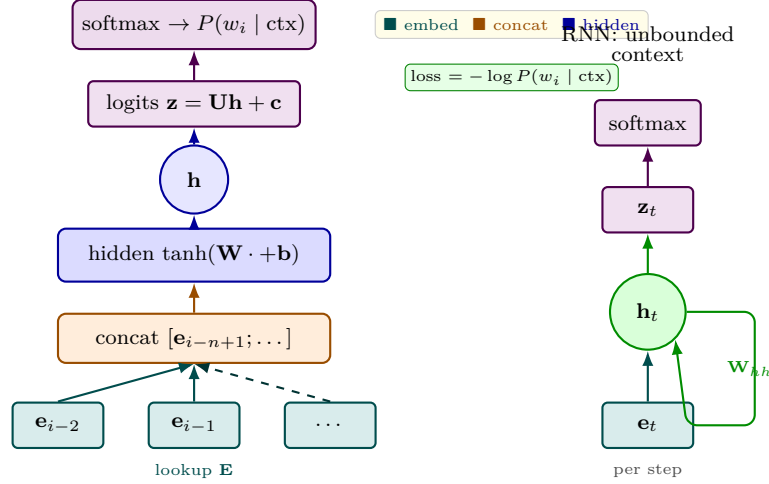


Figure 5.2: Feedforward NNLM (left) concatenates window embeddings through tanh to softmax; RNN LM (right) recurs $\mathbf{h}_t$ with $\mathbf{W}_{hh}$ for unbounded history. Colours: embedding teal, hidden blue/green, output violet.

Perplexity: the LM yardstick. Entropy $H = -\sum_w P^*(w) \log_2 P^*(w)$ measures bits per word under the true distribution P^* . We estimate cross-entropy on a test corpus w_1^N using our model Q :

$$H(Q) = -\frac{1}{N} \sum_{i=1}^N \log_2 Q(w_i | \text{history}), \quad \text{PP}(Q) = 2^{H(Q)} = \exp\left(-\frac{1}{N} \sum_{i=1}^N \ln Q(w_i | \text{history})\right). \quad (5.11)$$

Lower is better. Intuition: if Q assigns uniform $1/V$, then $H = \log_2 V$ and $\text{PP} = V$. A model with $\text{PP} = 50$ is as confused as if it chose uniformly among 50 words at each step. Perplexity is comparable only when V and tokenization are fixed; smoothing and neural models compete directly on this number.

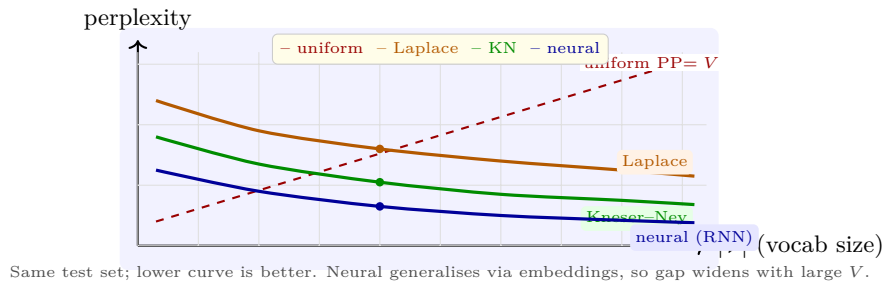


Figure 5.3: Perplexity versus vocabulary size (schematic). Uniform scales linearly. Laplace suffers most; Kneser–Ney is strong among count models; neural LMs (feedforward/RNN) achieve lowest perplexity because embeddings share statistical strength across similar histories.

5.4 From Scratch in Python

Listing 5.1: Interpolated Kneser–Ney (bigram) and perplexity. Pure Python, no extra deps.

```

1 from collections import Counter, defaultdict
2 import math
3
4 corpus = ["<s> the cat sat </s>", "<s> the cat ran </s>", "<s> the dog sat </s>"]
5 # counts
6 bigram_c = Counter(); unigram_c = Counter(); cont_counts = defaultdict(set)
7 for sent in corpus:
8     toks = sent.split()
9     for i, w in enumerate(toks):
10         unigram_c[w] += 1
11         if i > 0:
12             bigram_c[(toks[i-1], w)] += 1
13             cont_counts[w].add(toks[i-1])
14
15 # continuation probability
16 all_bigrams = len(bigram_c)
17 p_cont = {w: len(s)/all_bigrams for w, s in cont_counts.items()}
18 D = 0.75 # discount
19
20 def p_kn(prev, word):
21     c_bi = bigram_c[(prev, word)]
22     c_prev = unigram_c[prev]
23     if c_prev == 0:
24         return p_cont.get(word, 1e-8)
25     # distinct continuations of prev
26     uniq = sum(1 for (a, b) in bigram_c if a == prev)
27     lam = D * uniq / c_prev if c_prev else 0.0
28     discounted = max(c_bi - D, 0) / c_prev
29     return discounted + lam * p_cont.get(word, 1e-8)
30
31 print("P_KN(sat|cat) =", round(p_kn("cat", "sat"), 4))
32 print("P_KN(mat|cat) =", round(p_kn("cat", "mat"), 4)) # unseen > 0
33
34 # perplexity of a test sentence under KN
35 test = "<s> the cat sat </s>".split()
36 log_sum = 0.0
37 for i in range(1, len(test)):
38     p = p_kn(test[i-1], test[i])
39     log_sum += math.log(p + 1e-12) # ln
40 H_nats = -log_sum / (len(test)-1)
41 ppl = math.exp(H_nats) # = 2^{H_bits}
42 print(f"Perplexity = {ppl:.2f} (H={H_nats/math.log(2):.2f} bits)")
43 # compare Laplace bigram for reference
44 V = len(unigram_c)
45 def p_lap(prev, word, alpha=1.0):
46     return (bigram_c[(prev, word)] + alpha) / (unigram_c[prev] + alpha*V)
47 print("P_Lap(mat|cat) =", round(p_lap("cat", "mat"), 4))

```

Cross-entropy loss $-\log Q$ is the training objective; exponentiating it gives perplexity, so every gradient step

directly lowers PP on training data (validate on held-out data to avoid overfitting).

5.5 Chapter Notes

5.6 Exercises

- E5.1 **MLE and zeros.** Corpus: “<s> a b a”, “<s> a b”. Compute $P_{\text{MLE}}(b | a)$, $P_{\text{MLE}}(a | b)$, and $P(b, a | \text{<s>})$ via the chain rule. Show that any sentence containing the unseen bigram $(b, \text{<s>})$ gets probability 0 and explain why this is catastrophic for evaluation.
- E5.2 **Laplace vs. Good–Turing.** With $V = 10\,000$, history “cat” has $c(\text{cat}) = 1000$, $c(\text{cat}, \text{sat}) = 80$, $c(\text{cat}, \text{mat}) = 0$, and $N_1 = 4000$, $N_2 = 1500$ for bigrams. Compute $P_{\text{Lap}}(\text{sat} | \text{cat})$ and $P_{\text{Lap}}(\text{mat} | \text{cat})$ with $\alpha = 1$. Then compute the Good–Turing r^* for $r = 0$ and $r = 1$ and discuss which method wastes more mass on unseen bigrams.
- E5.3 **Kneser–Ney continuation.** In a corpus, “Francisco” appears 100 times but only after “San”, while “dog” appears 100 times after 80 distinct words. With $|\{(u, v) : c(u, v) > 0\}| = 10\,000$, compute P_{cont} for each and use Eq. (5.6) with $D = 0.75$, $c(\text{San}, \text{Francisco}) = 100$, $c(\text{San}) = 100$, $c(\text{the}, \text{dog}) = 5$, $c(\text{the}) = 1000$ to show $P_{\text{KN}}(\text{Francisco} | \text{San}) \gg P_{\text{KN}}(\text{Francisco} | \text{the})$ but $P_{\text{cont}}(\text{dog}) \gg P_{\text{cont}}(\text{Francisco})$.
- E5.4 **Neural LM and perplexity.** A feedforward LM with window $n = 3$, $d = 50$, $V = 20\,000$, hidden $h = 200$ has embedding matrix $\mathbf{E} \in \mathbb{R}^{d \times V}$ and $\mathbf{W} \in \mathbb{R}^{h \times (n-1)d}$, $\mathbf{U} \in \mathbb{R}^{V \times h}$. Count parameters. For test sentence of $N = 10$ words where the model assigns $\log_2 Q = -3$ bits each, compute H and $\text{PP} = 2^H$. If a second model gives $\text{PP} = 45$, which is better and why?
- E5.5 **Implement and compare.** Extend the Python listing to (a) compute trigram P_{KN} by recursing to bigram P_{KN} , (b) report perplexity on a held-out sentence for Laplace vs. KN, and (c) replace count lookup with a tiny PyTorch feedforward NNLM ($d = 16$, $h = 32$) trained by cross-entropy and compare perplexities. Explain why the neural model wins on unseen n -grams.

Chapter 6

Sequence-to-Sequence Learning: Encoder-Decoder, Attention, and Beam Search

“To translate you must first understand, then generate—but how much can you squeeze through a single vector?” — This chapter compresses a sentence into a bottleneck, then learns to look back.

From zero: no seq2seq background assumed. We start from translation as mapping one sequence to another of different length, build the encoder-decoder RNN, diagnose its bottleneck, fix it with attention (Bahdanau and Luong), and decode with beam search. Every formula is motivated by a picture first.

Learning Outcomes

After this chapter you will be able to:

- (L1) formulate machine translation as seq2seq and explain the fixed-vector bottleneck;
- (L2) write encoder and decoder recurrences and the factorization $P(\mathbf{y} \mid \mathbf{x})$;
- (L3) derive attention weights $\alpha_{ij} = \text{softmax}(\text{score})$ and the context vector $\mathbf{c}_i$ in Bahdanau and Luong variants;
- (L4) run beam search with width k and contrast it with greedy decoding;
- (L5) describe BLEU and teacher forcing and why they matter for Seq2Seq training.

6.1 Translation as Seq2Seq: The Bottleneck Problem

Intuition. Translation maps “*le chat dort*” $\rightarrow$ “*the cat sleeps*”—input and output have different lengths, no word-to-word alignment is given, and order may change. Seq2seq treats it as: read the whole French sentence,

remember its meaning, then write English one word at a time. The earliest solution compressed everything into one vector $\mathbf{c}$: the encoder’s last hidden state.

Why this hurts. Imagine summarizing a 30-word sentence into a single 512-dimensional vector and then reproducing it perfectly. Information is lost—the bottleneck. Long sentences degrade first; the decoder’s first steps see the same $\mathbf{c}$ as its last steps, with no way to re-inspect the source. Performance on WMT translation visibly drops past 20 tokens without a fix.

Example. Source $\mathbf{x} = (x_1, x_2, x_3) = (\text{le}, \text{chat}, \text{dort})$, target $\mathbf{y} = (y_1, y_2, y_3) = (\text{the}, \text{cat}, \text{sleeps})$. We need a model for $P(\mathbf{y} | \mathbf{x}) = \prod_{t=1}^{T_y} P(y_t | y_{<t}, \mathbf{x})$ that works for any $T_y \neq T_x$.

6.2 The Encoder-Decoder Architecture

Formal definition. Let source embeddings $\mathbf{e}_t^x$ and target embeddings $\mathbf{e}_t^y$. The *encoder* is an RNN (vanilla, GRU, or LSTM):

$$\mathbf{h}_t = f_{\text{enc}}(\mathbf{e}_t^x, \mathbf{h}_{t-1}), \quad t = 1, \dots, T_x, \quad \mathbf{c} = \mathbf{h}_{T_x}, \quad (6.1)$$

with variants: bidirectional $\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$. The *decoder* generates autoregressively:

$$\mathbf{s}_t = f_{\text{dec}}(\mathbf{e}_{t-1}^y, \mathbf{s}_{t-1}, \mathbf{c}), \quad P(y_t | y_{<t}, \mathbf{x}) = \text{softmax}(W_o \mathbf{s}_t + \mathbf{b}_o)_{y_t}, \quad (6.2)$$

with $\mathbf{s}_0 = \mathbf{c}$ and $y_0 = \langle s \rangle$. Training maximizes $\sum_{(\mathbf{x}, \mathbf{y})} \log P(\mathbf{y} | \mathbf{x})$ via teacher forcing (feed gold y_{t-1}). Fig. 6.1 shows the bottleneck.

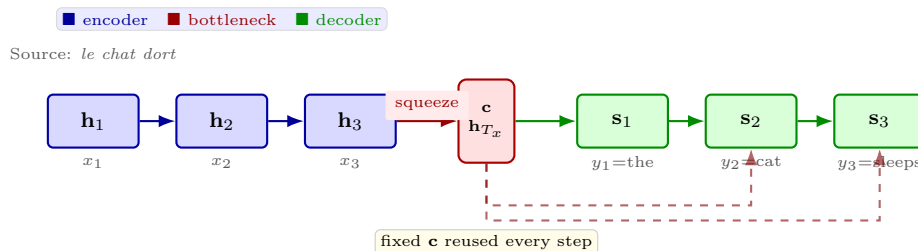


Figure 6.1: Encoder-decoder bottleneck. The encoder compresses $\mathbf{x}$ into a single vector $\mathbf{c} = \mathbf{h}_{T_x}$ (red); the decoder sees only $\mathbf{c}$ at every step. Long inputs lose detail—attention will let the decoder re-read all $\mathbf{h}_j$.

Without attention the model must memorize word order, agreement, and alignment inside $\mathbf{c}$ alone, which fails for $T_x \gg 10$.

6.3 Attention: Letting the Decoder Look Back

Intuition. A human translator does not memorize the French sentence and then close the book. She glances back at relevant source words while writing each English word. Attention gives the decoder the same ability: at step i it computes a weighted average of *all* encoder states, focusing on the useful positions.

Formal mechanism. Keep all encoder states $\mathbf{h}_1, \dots, \mathbf{h}_{T_x}$. At decoder step i with state $\mathbf{s}_{i-1}$:

$$e_{ij} = \text{score}(\mathbf{s}_{i-1}, \mathbf{h}_j) \quad \text{alignment score} \quad (6.3)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})} = \text{softmax}_j(e_{ij}) \quad \text{attention weight, } \sum_j \alpha_{ij} = 1 \quad (6.4)$$

$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j \quad \text{context vector (dynamic)} \quad (6.5)$$

$$\mathbf{s}_i = f_{\text{dec}}(\mathbf{e}_{i-1}^y, \mathbf{s}_{i-1}, \mathbf{c}_i), \quad P(y_i | y_{<i}, \mathbf{x}) = \text{softmax}(W_o[\mathbf{s}_i; \mathbf{c}_i]). \quad (6.6)$$

Two scoring variants:

$$\text{Bahdanau: } e_{ij} = \mathbf{v}_a^\top \tanh(W_a \mathbf{s}_{i-1} + U_a \mathbf{h}_j), \quad \text{Luong: } e_{ij} = \mathbf{s}_{i-1}^\top W_a \mathbf{h}_j \text{ (general) or } \mathbf{s}_{i-1}^\top \mathbf{h}_j \text{ (dot)}. \quad (6.7)$$

Bahdanau (additive) feeds $[\mathbf{s}_i; \mathbf{c}_i]$ into the output; Luong (multiplicative) often uses $\tilde{\mathbf{s}}_i = \tanh(W_c[\mathbf{s}_i; \mathbf{c}_i])$. Fig. 6.2 visualizes α_{ij} .

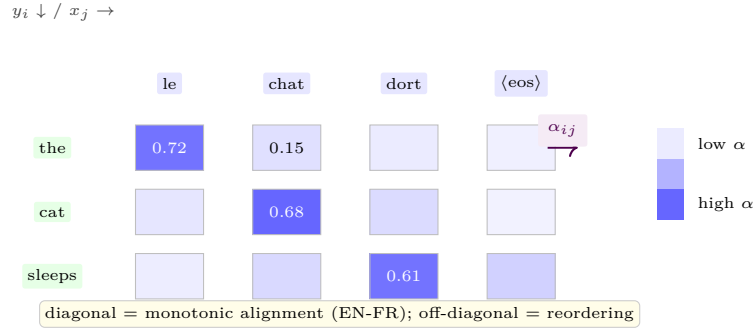


Figure 6.2: Attention heatmap α_{ij} for *le chat dort* $\rightarrow$ *the cat sleeps*. Each row sums to 1 (softmax). Bright diagonal shows the model attends to the corresponding source word; context $\mathbf{c}_i$ is the α -weighted sum of all $\mathbf{h}_j$.

Attention removes the bottleneck: $\mathbf{c}_i$ is recomputed each step, gradients flow directly to the relevant $\mathbf{h}_j$, and alignment becomes interpretable.

6.4 Decoding with Beam Search and Training

Greedy vs. beam search. Greedy decoding picks $\hat{y}_t = \arg \max P(y_t | \hat{y}_{<t}, \mathbf{x})$ —locally optimal, globally myopic. Beam search keeps the top- k hypotheses (beam width k), expanding each by $|V|$ words and pruning to k best by log-probability:

$$\text{score}(\mathbf{y}_{1:t}) = \sum_{i=1}^t \log P(y_i | y_{<i}, \mathbf{x}), \quad \mathcal{B}_t = \text{top}_k\{\text{score}(\mathbf{y}) \mid \mathbf{y} \in \mathcal{B}_{t-1} \times V\}. \quad (6.8)$$

With $k = 1$ this is greedy; $k = 5$ – 10 is typical. Hypotheses ending in $\langle \text{eos} \rangle$ are complete; we normalize by length $(1/T)^\alpha$ to avoid short-sequence bias. Fig. 6.3 traces $k = 2$.

Training and BLEU. Training uses teacher forcing and cross-entropy on $P(\mathbf{y} | \mathbf{x})$. At test time we report BLEU: geometric mean of n -gram precisions ($n = 1..4$) times brevity penalty $\text{BP} = \exp(1 - r/c)$ if $c < r$

(candidate shorter than reference). BLEU correlates with human judgment but penalizes paraphrases; it is not differentiable, so we train on likelihood and evaluate on BLEU.

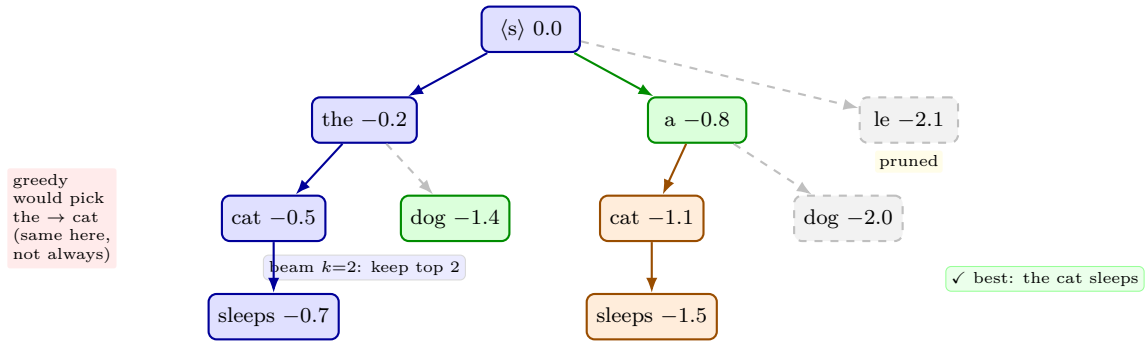


Figure 6.3: Beam search with width $k = 2$. Each level expands $k \times |V|$ candidates, keeps the k highest $\sum \log P$. Pruned hypotheses (dashed gray) are discarded. Completed hypotheses are length-normalized before final selection; greedy ($k = 1$) can miss the global optimum.

Listing 6.1: Attention weights and beam search (width k) for seq2seq. Bahdanau scores shown; swap in Luong dot-product easily.

```

1 import numpy as np
2
3 def attention_scores(s_prev, H, Wa, Ua, va):
4     """Bahdanau scores: s_prev (d), H (Tx,d) -> e (Tx,)"""
5     # e_j = v_a^T tanh(W_a s + U_a h_j)
6     proj_s = Wa @ s_prev          # (d,)
7     proj_H = H @ Ua.T            # (Tx, d)
8     e = np.tanh(proj_H + proj_s) @ va # (Tx,)
9     alpha = np.exp(e) / np.sum(np.exp(e)) # softmax Eq.12
10    c = alpha @ H                  # (d,) context vector Eq.13
11    return e, alpha, c
12
13 def beam_search(decode_step, enc_states, beam_width=2, max_len=10):
14     """decode_step(beam_seq, context) -> dict{word: logprob}."""
15     beams = [[([0], 0.0)] # (token_ids, logprob); 0 = <s>
16     completed = []
17     for t in range(max_len):
18         candidates = []
19         for seq, score in beams:
20             if seq[-1] == 1: # <eos> id=1, already complete
21                 completed.append((seq, score / len(seq)**0.6)) # length norm
22                 continue
23             # recompute attention context for this hypothesis
24             # (simplified: use last decoder state approximated by mean)
25             logprobs = decode_step(seq, enc_states) # {wid: logp}
26             for wid, lp in logprobs.items():
27                 candidates.append((seq + [wid], score + lp))
28         if not candidates:
29             break
30         candidates.sort(key=lambda x: x[1], reverse=True)
31         beams = candidates[:beam_width]
32         if len(completed) >= beam_width:
33             break
34     completed.extend((s, sc / len(s)**0.6) for s, sc in beams if s[-1]==1)

```

```

35     best = max(completed or beams, key=lambda x: x[1])
36     return best # (best_seq, best_score)
37
38 # Tiny demo: Tx=3, d=4
39 np.random.seed(0)
40 H = np.random.randn(3, 4)
41 s = np.random.randn(4)
42 Wa, Ua = np.random.randn(4,4), np.random.randn(4,4)
43 va = np.random.randn(4)
44 e, alpha, c = attention_scores(s, H, Wa, Ua, va)
45 print(f"alpha={np.round(alpha,3)} sum={alpha.sum():.2f}") # sums to 1
46 print(f"context shape {c.shape}")

```

6.5 Chapter Notes

Sutskever, Vinyals and Le (2014) and Cho et al. (2014) introduced encoder-decoder RNNs for machine translation. Bahdanau, Cho and Bengio (2015) added additive attention to break the bottleneck, followed by Luong, Pham and Manning (2015) with multiplicative variants. Beam search analysis and length normalization tricks are standard in neural MT; BLEU was defined by Papineni et al. (2002). Attention paved the way for the Transformer.

6.6 Exercises

- E6.1 **Bottleneck intuition.** Give a 25-word English sentence that a fixed-vector encoder-decoder is likely to mistranslate and explain which information the bottleneck would discard. How does attention avoid this loss?
- E6.2 **Attention by hand.** Let $\mathbf{s} = [1, 0]$, $\mathbf{h}_1 = [1, 1]$, $\mathbf{h}_2 = [-1, 1]$, Luong dot score $e_j = \mathbf{s}^\top \mathbf{h}_j$. Compute e_1, e_2 , then $\alpha_1, \alpha_2 = \text{softmax}(e)$, then $\mathbf{c} = \sum_j \alpha_j \mathbf{h}_j$. Repeat with Bahdanau where $W_a = U_a = I$, $\mathbf{v}_a = [1, 1]$.
- E6.3 **Bahdanau vs. Luong.** State one advantage of Bahdanau additive scoring (hint: dimensions) and one of Luong dot-product (hint: speed). When $\mathbf{s}$ and $\mathbf{h}_j$ have different dimensions, which variants apply without projection?
- E6.4 **Beam search trace.** Vocabulary $\{a, b, \langle \text{eos} \rangle\}$, $k = 2$. At $t=1$: $P(a) = 0.5, P(b) = 0.3, P(\text{eos}) = 0.2$. From a : $P(a) = 0.1, P(b) = 0.1, P(\text{eos}) = 0.8$; from b : $P(a) = 0.6, P(b) = 0.2, P(\text{eos}) = 0.2$. Run beam search for two steps, list kept beams with scores, and compare the best beam to greedy.
- E6.5 **BLEU and teacher forcing.** Compute BLEU-2 for candidate “the cat cat” vs. reference “the cat sleeps” with brevity penalty. Then explain *exposure bias* from teacher forcing and one mitigation (scheduled sampling or minimum-risk/BLEU training).

Chapter 7

Pretraining: BERT, GPT, Fine-Tuning, and Prompting

“Pretraining teaches a model the language; fine-tuning teaches it the job; prompting tells it the job in words.” — This chapter covers the paradigm that reshaped NLP: train once on raw text, reuse everywhere.

From zero: no pretraining background assumed. We start from the everyday idea of learning general knowledge before specializing, then ask why predicting hidden words yields useful language understanding. We build to BERT and GPT objectives, fine-tuning, and prompting from intuition to formal equations. Every formula is motivated by a picture first.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain the pretrain–then–fine-tune paradigm and why self-supervision scales;
- (L2) write the BERT masked language model and next sentence prediction objectives;
- (L3) contrast bidirectional and autoregressive pretraining and state when each helps;
- (L4) describe fine-tuning with [CLS], prompting as conditioning, and in-context learning;
- (L5) explain adapter and LoRA parameter-efficient tuning at an intuitive level.

7.1 Why Pretrain? The Paradigm

Intuition. Humans learn language for years before learning a specific job like medical diagnosis. NLP does the same: first read billions of words with no labels, then specialize with a small labeled set. Raw text is abundant; labeled data is scarce. Pretraining turns the abundant signal into reusable weights.

Imagine filling blanks in a sentence: “The cat ____ on the mat.” To guess “sat” you need syntax, semantics, and world knowledge. A model that gets good at this blank-filling has implicitly learned grammar and facts. Afterward, adapting it to sentiment or question answering needs far less supervision.

Formal view. A pretrained language model (PLM) learns parameters θ by self-supervision on unlabeled corpus $\mathcal{C} = \{\mathbf{x}^{(i)}\}_{i=1}^N$. Then for downstream task $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}$ we either update θ (fine-tuning) or keep θ frozen and shape the input (prompting). The pretraining objective needs no human labels: the text itself provides supervision by hiding and predicting parts.

Why not predict left-to-right only? That gives only left context. To represent a word fully we want both sides: “bank” in “river bank” versus “bank account” is disambiguated by words on both sides. This motivates masked language modeling.

7.2 BERT: Masked Language Modeling and NSP

Intuition. BERT (Devlin et al., 2019) plays a cloze test: randomly cover 15% of tokens with [MASK] and ask the Transformer encoder to recover them using *all* surrounding words (left and right). Because the encoder has no causal restriction, each position attends to every other position—the context is bidirectional. A second task, next sentence prediction (NSP), teaches sentence relationships: given sentence pair (A, B) , predict whether B actually follows A .

Formal: MLM objective. Let $\mathbf{x} = (x_1, \dots, x_T)$ and $\mathcal{M} \subset \{1, \dots, T\}$ be masked positions ($|\mathcal{M}| \approx 0.15T$). Let $\tilde{\mathbf{x}}$ be $\mathbf{x}$ with x_i replaced by [MASK] (80%), a random token (10%), or kept (10%) for $i \in \mathcal{M}$. The encoder produces $\mathbf{h}_i = f_\theta(\tilde{\mathbf{x}})_i$. Then

$$\mathcal{L}_{\text{MLM}}(\theta) = -\mathbb{E}_{\mathbf{x} \sim \mathcal{C}} \sum_{i \in \mathcal{M}} \log P_\theta(x_i | \tilde{\mathbf{x}}), \quad P_\theta(x_i | \tilde{\mathbf{x}}) = \text{softmax}(\mathbf{W}_o \mathbf{h}_i + \mathbf{b}_o)_{x_i}. \quad (7.1)$$

Only masked positions contribute to the loss; the model cannot cheat by copying the input because the input at $i \in \mathcal{M}$ is hidden. Bidirectionality is key: $\mathbf{h}_i$ depends on $\tilde{\mathbf{x}}_{1:T}$, not just $\tilde{\mathbf{x}}_{<i}$.

Formal: NSP objective. Input is $\mathbf{x}_{\text{pair}} = [[\text{CLS}] A [\text{SEP}] B [\text{SEP}]]$. Let $\mathbf{h}_{\text{CLS}}$ be the final hidden state of [CLS]. NSP is binary classification:

$$\mathcal{L}_{\text{NSP}}(\theta) = -\mathbb{E}_{(A,B,y)} [y \log \sigma(\mathbf{w}^\top \mathbf{h}_{\text{CLS}}) + (1-y) \log(1 - \sigma(\mathbf{w}^\top \mathbf{h}_{\text{CLS}}))], \quad (7.2)$$

where $y = 1$ if B is the true next sentence, else 0. Total pretraining loss is $\mathcal{L}_{\text{BERT}} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$. Later work shows NSP contributes less than MLM, but the [CLS] representation trained this way becomes a sentence-level vector for fine-tuning.

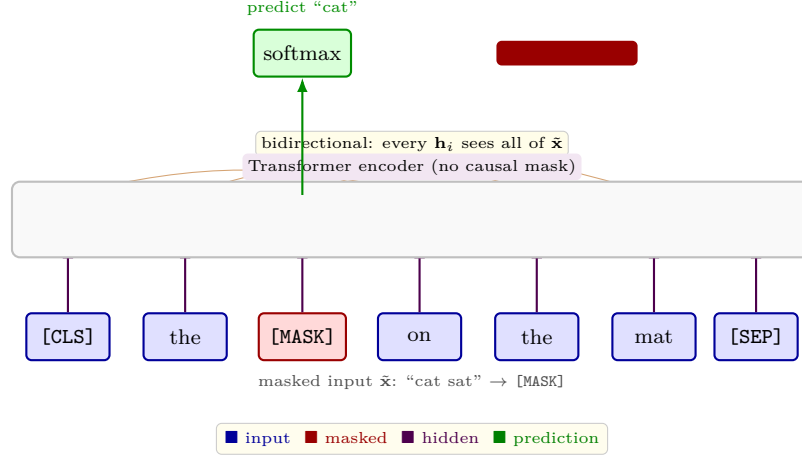


Figure 7.1: BERT masked language modeling. Input tokens are partially masked; the Transformer encoder attends bidirectionally (orange arcs) to produce hidden states $\mathbf{h}_i$. Only masked positions (orange highlight) predict the original token via a softmax; other positions contribute no MLM loss.

7.3 GPT: Autoregressive (Causal) Pretraining

Intuition. GPT (Radford et al., 2018) learns like writing left-to-right: given “The cat sat”, predict what comes next, one word at a time. The model never sees the future during training, so at test time it can generate fluently by feeding its own predictions back.

Formal: causal LM objective. For sequence $\mathbf{x} = (x_1, \dots, x_T)$,

$$\mathcal{L}_{\text{LM}}(\theta) = -\mathbb{E}_{\mathbf{x} \sim \mathcal{C}} \sum_{t=1}^T \log P_{\theta}(x_t | x_{<t}), \quad P_{\theta}(x_t | x_{<t}) = \text{softmax}(\mathbf{W}_o \mathbf{h}_{t-1} + \mathbf{b}_o)_{x_t}, \quad (7.3)$$

where $\mathbf{h}_{t-1}$ is the Transformer decoder hidden state at position $t-1$. Causality is enforced by a triangular attention mask $M_{ij} = -\infty$ for $j > i$, so position i attends only to $j \leq i$:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}} + M\right)V. \quad (7.4)$$

Without M the model would see x_t when predicting it—trivial and useless for generation. Fig. 7.2 contrasts the mask with BERT’s full attention.

BERT is bidirectional but non-generative (needs masking trick to avoid leakage); GPT is unidirectional but naturally generative. Downstream, BERT excels at understanding, GPT at generation; modern models often blend both.

7.4 Fine-Tuning, Prompting, and Efficient Adaptation

Fine-tuning with [CLS]. After pretraining, add a task head atop $\mathbf{h}_{\text{CLS}}$. For classification with C classes,

$$P(y | \mathbf{x}) = \text{softmax}(\mathbf{W}_c \mathbf{h}_{\text{CLS}} + \mathbf{b}_c)_y, \quad \mathcal{L}_{\text{FT}} = -\sum_{i=1}^{|\mathcal{D}|} \log P(y^{(i)} | \mathbf{x}^{(i)}). \quad (7.5)$$

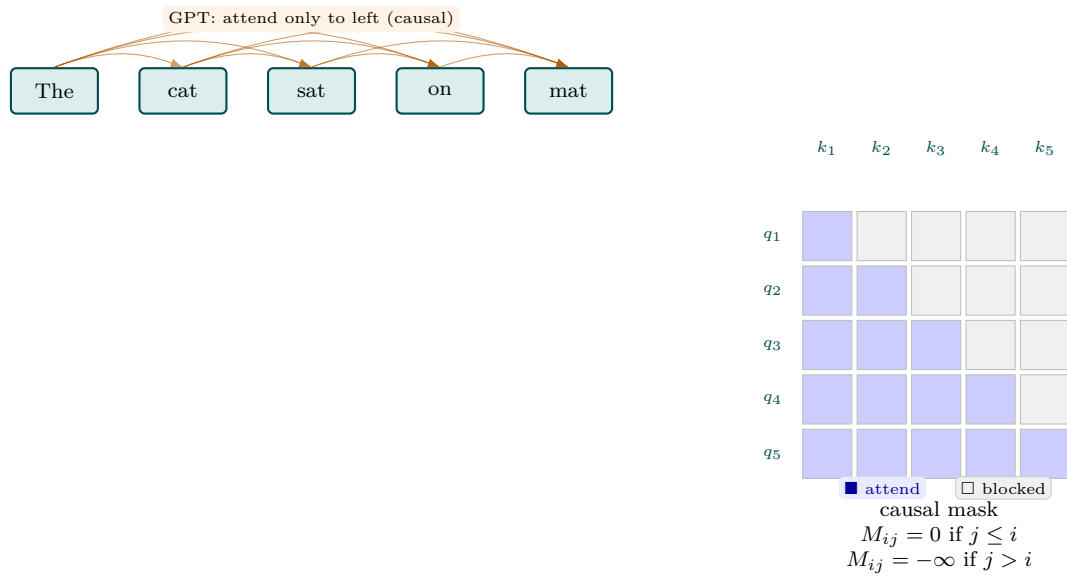


Figure 7.2: GPT causal language modeling. Left: each token can only attend to itself and earlier tokens (orange arcs). Right: the 5×5 attention mask is lower-triangular; gray cells are set to $-\infty$ before softmax so future tokens get zero weight. BERT would fill the whole matrix blue.

All parameters (including the PLM) are updated with a small learning rate (2×10^{-5}), or optionally only the head is trained (feature extraction). The same PLM spawns many fine-tuned copies, one per task.

Prompting as conditioning. Instead of updating θ , wrap $\mathbf{x}$ in a natural-language template and let the frozen PLM complete it. *Discrete prompts* use a hand-written pattern, e.g. sentiment: “Review: *Great film!* Sentiment: [MASK]” $\rightarrow$ model predicts “positive”. Formally we compute $P(y \mid \text{prompt}(\mathbf{x}))$ without gradient steps.

In-context learning (Brown et al., 2020) concatenates k demonstrations before the query: $\mathbf{x}_{\text{ctx}} = [(x^{(1)}, y^{(1)}), \dots, (x^{(k)}, y^{(k)}), x]$. No weight update occurs; the model conditions on the context via attention alone. *Instruction tuning* fine-tunes once on a mixture of instruction–response pairs (e.g. FLAN, InstructGPT), so at test time a plain instruction like “Translate to French:” works zero-shot.

Parameter-efficient tuning. Full fine-tuning copies 110M–175B parameters per task. Two efficient alternatives keep the base frozen: *Adapter*: insert a small bottleneck $\mathbf{h} \leftarrow \mathbf{h} + \mathbf{W}_{\text{up}} \sigma(\mathbf{W}_{\text{down}} \mathbf{h})$ inside each Transformer block; only $\mathbf{W}_{\text{down}} \in \mathbb{R}^{m \times d}$, $\mathbf{W}_{\text{up}} \in \mathbb{R}^{d \times m}$ with $m \ll d$ are trained ($\sim 1\text{--}3\%$ params). *LoRA* (Hu et al., 2022): freeze $\mathbf{W}_0$ and learn a low-rank delta $\Delta \mathbf{W} = \mathbf{B} \mathbf{A}$ with $\mathbf{B} \in \mathbb{R}^{d \times r}$, $\mathbf{A} \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$:

$$\mathbf{h} = \mathbf{W}_0 \mathbf{x} + \frac{\alpha}{r} \mathbf{B} \mathbf{A} \mathbf{x}, \quad (7.6)$$

used for query/value projections. Only $\mathbf{B}, \mathbf{A}$ are stored per task—megabytes, not gigabytes—yet performance nears full fine-tuning.

Listing 7.1: BERT mask filling and prompting with Hugging Face `transformers`. The same checkpoint does cloze and zero-shot classification via prompts.

```

1 from transformers import pipeline, AutoTokenizer, AutoModelForMaskedLM
2 import torch

```

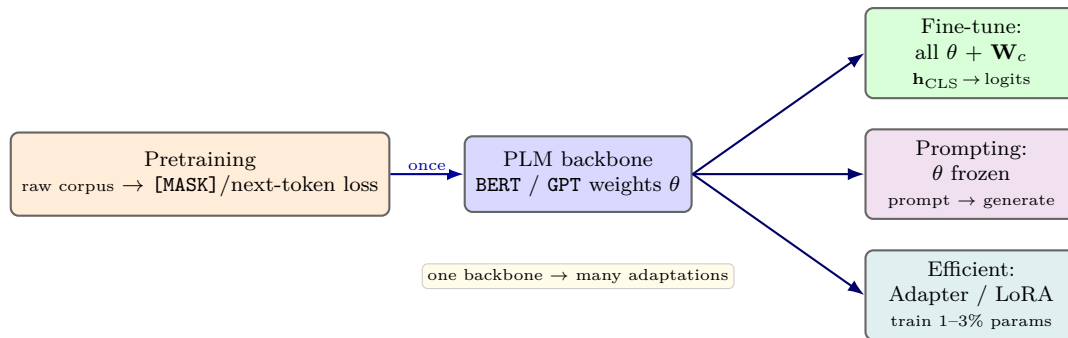


Figure 7.3: Pretrain-then-adapt pipeline. A single self-supervised pretraining run yields a reusable PLM. Three adaptation paths reuse it: full fine-tuning (update all weights via a [CLS] head), prompting (freeze θ , condition via text), and parameter-efficient tuning (freeze θ , train small adapter/LoRA deltas).

```

3
4 # --- 1. BERT masked LM cloze ---
5 fill = pipeline("fill-mask", model="bert-base-uncased")
6 print(fill("The cat [MASK] on the mat."))
7 # [{'token_str': 'sat', 'score': 0.42}, {'token_str': 'lay', ...}]
8
9 # --- 2. Manual MLM with logits ---
10 tok = AutoTokenizer.from_pretrained("bert-base-uncased")
11 model = AutoModelForMaskedLM.from_pretrained("bert-base-uncased")
12 inputs = tok("The capital of France is [MASK].", return_tensors="pt")
13 with torch.no_grad():
14     logits = model(**inputs).logits # [1, T, V]
15 mask_id = tok.mask_token_id
16 pred_id = logits[0, (inputs.input_ids == mask_id).nonzero(as_tuple=True)[1]].argmax(-1)
17 print("Predicted:", tok.decode(pred_id)) # -> 'paris'
18
19 # --- 3. Prompting as zero-shot classification (no weight update) ---
20 zeroshot = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")
21 print(zeroshot("I loved the movie, it was fantastic!", candidate_labels=["positive", "
    negative"]))
22 # {'labels': ['positive', 'negative'], 'scores': [0.98, 0.02]}
23
24 # --- 4. In-context few-shot prompt for GPT-style model ---
25 prompt = """Review: This film was dull. Sentiment: negative
26 Review: A masterpiece! Sentiment: positive
27 Review: It was okay, not great. Sentiment:"""
28 # feed 'prompt' to an autoregressive LM; it continues with 'negative'/'positive'
29 print(prompt)

```

7.5 Chapter Notes

BERT (Devlin et al., 2019) introduced MLM+NSP; RoBERTa (Liu et al., 2019) showed NSP is expendable and longer training matters. GPT-2/3 (Radford 2019; Brown 2020) scaled causal LM to few-shot prompting. Parameter-efficient tuning surveys: adapters (Houlsby et al., 2019), LoRA (Hu et al., 2022). Instruction tuning was popularized by FLAN (Wei et al., 2022) and InstructGPT (Ouyang et al., 2022).

7.6 Exercises

- E7.1 **MLM objective.** A 10-token sentence has 2 masked positions. Write Eq. (7.1) for this sentence and explain why loss on unmasked positions would let the model copy inputs. How does the 80/10/10 masking mix discourage reliance on [MASK] at fine-tuning time?
- E7.2 **Bidirectional vs. causal.** Give a word-sense example where BERT’s bidirectional context succeeds but a left-only model fails (e.g. “bank” disambiguated by a word to its right). Write the attention reach for each model at the target position.
- E7.3 **Causal mask.** For $T = 4$, write the 4×4 matrix M in Eq. (7.4) with entries 0 or $-\infty$. If a bug leaves $M_{2,4} = 0$, which token can illegally see the future and what does this do to next-token prediction?
- E7.4 **Fine-tune vs. prompt.** You have 32 labeled sentiment examples. Compare fine-tuning a [CLS] head (Eq. (7.5)) versus in-context prompting with $k = 8$ examples: which updates weights, which scales with context length, and which needs more labeled data? When would you choose LoRA instead?
- E7.5 **LoRA arithmetic.** A query projection has $\mathbf{W}_0 \in \mathbb{R}^{768 \times 768}$. LoRA uses $r = 8$. How many trainable parameters does $\Delta \mathbf{W} = \mathbf{B} \mathbf{A}$ add? Compare to full fine-tuning and to an adapter bottleneck $m = 64$ in each of 12 layers. Why does $r \ll d$ still work?

Chapter 8

Generation: Sampling, Alignment, and Evaluation

“A language model knows how to continue; generation decides which continuation to show, and evaluation decides whether it was any good.” — This chapter turns next-token probabilities into text and judgement.

From zero: no generation background assumed. We start from a model that outputs a distribution over the next word and ask: why picking the best word every time fails, how to inject controlled randomness, how to align outputs to human preferences with RLHF, and how BLEU/ROUGE score generated text. Every formula is motivated by a picture first.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain why greedy decoding is locally optimal but globally suboptimal and repetitive;
- (L2) write sampling formulas for temperature, top- k , and top- p (nucleus) and trade diversity vs. quality;
- (L3) describe the RLHF pipeline (SFT, reward model, PPO) and the DPO shortcut;
- (L4) compute BLEU with n-gram precision, brevity penalty, and ROUGE recall including ROUGE-L;
- (L5) implement nucleus sampling and BLEU in Python and choose metrics for a task.

8.1 From Probabilities to Text: Why Greedy Fails

Intuition. A language model gives $P(y_t \mid y_{<t}, x)$ for every vocab item. Greedy decoding picks $\hat{y}_t = \arg \max_y P(y \mid y_{<t})$ at each step. This is like always taking the steepest uphill step: you reach a local peak but miss the mountain. Worse, language is multimodal: after “I love”, both “cats” and “dogs” are good; greedy commits to one

path and cannot backtrack if later words become awkward. Empirically, greedy produces bland, repetitive text (“I think I think I think”) because high-frequency phrases dominate each local choice.

Formal setup. Let logits $\mathbf{z} \in \mathbb{R}^{|V|}$ produce $P(y = i \mid \text{ctx}) = \text{softmax}(\mathbf{z})_i$. Greedy decodes

$$y_t^* = \arg \max_i \log P(y_t = i \mid y_{<t}, x), \quad \mathbf{y}^* = (y_1^*, \dots, y_T^*). \quad (8.1)$$

Beam search generalises greedy: keep top- B hypotheses, scoring $\sum_t \log P(y_t \mid y_{<t})$. With $B = 1$ it is greedy; with larger B it approximates $\arg \max_{\mathbf{y}} P(\mathbf{y} \mid x)$ but still maximises likelihood, not human preference, and remains deterministic.

Example. Prompt: “The cat sat on the”. Model: $P(\text{mat} \mid \cdot) = 0.4$, $P(\text{roof} \mid \cdot) = 0.35$, $P(\text{chair} \mid \cdot) = 0.25$. Greedy picks “mat” every time, so 100 continuations are identical. A human wants variety: sometimes roof, sometimes chair, sometimes mat.

8.2 Sampling: Temperature, Top- k , Top- p

Intuition. Sampling treats the distribution as a menu: temperature sharpens or flattens it, top- k keeps only k dishes, top- p keeps enough dishes to cover p of the appetite. All aim to balance *quality* (stay on high-probability words) and *diversity* (allow surprises).

Temperature scaling. Divide logits by $\tau > 0$ before softmax:

$$P_\tau(y = i \mid \text{ctx}) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}. \quad (8.2)$$

$\tau \rightarrow 0$ sharpens to argmax (one-hot); $\tau = 1$ is the original; $\tau > 1$ flattens toward uniform. Low τ is safe but dull; high τ is creative but incoherent.

Top- k and nucleus (top- p). Let sorted $P_{(1)} \geq P_{(2)} \geq \dots$. Top- k keeps $\mathcal{V}_k = \{i : \text{rank}(i) \leq k\}$ and renormalises:

$$P_i^{(k)} = \frac{P_i \cdot \mathbf{1}[i \in \mathcal{V}_k]}{\sum_{j \in \mathcal{V}_k} P_j}. \quad (8.3)$$

Top- p (nucleus) keeps the *smallest* set with cumulative mass $\geq p$:

$$\mathcal{V}_p = \text{smallest } S \text{ s.t. } \sum_{i \in S} P_{(i)} \geq p, \quad P_i^{(p)} = \frac{P_i \cdot \mathbf{1}[i \in \mathcal{V}_p]}{\sum_{j \in \mathcal{V}_p} P_j}. \quad (8.4)$$

Fixed k wastes capacity when the distribution is peaked (top-1 suffices) and truncates too aggressively when flat; p adapts: keep 2 words when confident, 50 when uncertain. Fig. 8.1 visualises all three.

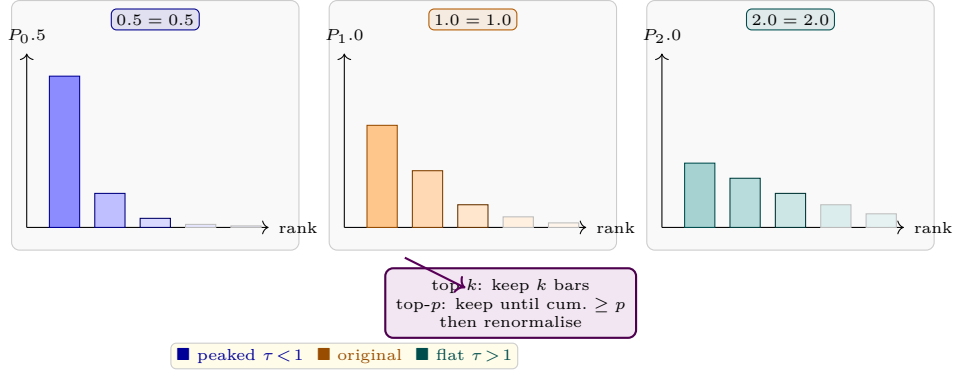


Figure 8.1: Temperature scaling and truncation. Low τ peaks on the top token (near-greedy); high τ spreads mass. Top- k cuts after k ranks regardless of shape; top- p cuts when cumulative mass reaches p , adapting to peaked vs. flat distributions.

8.3 Aligning to Humans: RLHF and DPO

Intuition. Likelihood rewards fluent text, not helpful or harmless text. Humans prefer answers that are truthful, concise, and refuse harmful requests, but the pre-training corpus contains all styles. RLHF teaches the model *which* fluent continuations humans actually want by learning a reward function from comparisons and then optimising against it.

RLHF pipeline. Three stages (Fig. 8.2):

1. **SFT**: fine-tune base LM on high-quality demonstrations (x, y^*) with cross-entropy.
2. **Reward model (RM)**: collect human preference pairs (x, y_w, y_l) where $y_w \succ y_l$. Train $r_\phi(x, y)$ with Bradley-Terry loss:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]. \quad (8.5)$$

3. **RL (PPO)**: treat LM π_θ as policy, maximise

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [r_\phi(x, y)] - \beta \text{KL}[\pi_\theta \| \pi_{\text{SFT}}], \quad (8.6)$$

via clipped PPO to stay near SFT. KL penalty prevents reward hacking (gibberish that fools r_ϕ).

DPO intuition. PPO is heavy (rollouts, value network). DPO observes that the optimal policy for Eq. (8.6) satisfies $r^* \propto \log \frac{\pi^*}{\pi_{\text{ref}}}$; substitute into Eq. (8.5) to train π_θ *directly* on preferences:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]. \quad (8.7)$$

No reward model, no sampling loop; DPO is supervised learning on pairs with an implicit KL.

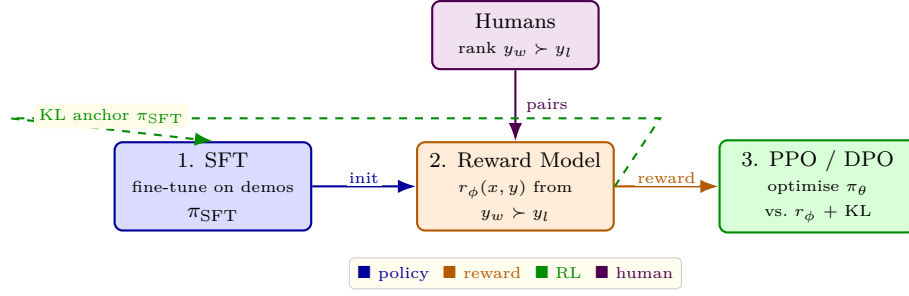


Figure 8.2: RLHF pipeline. SFT gives a helpful base; humans provide preference pairs to train a reward model r_ϕ ; PPO (or DPO) pushes π_θ to maximise r_ϕ while staying close to π_{SFT} via KL. DPO collapses steps 2–3 into one loss.

8.4 Evaluation: BLEU and ROUGE

Intuition. How to score a generated sentence against references? Count overlapping word chunks (n -grams). Precision asks “of what you wrote, how much was correct?” Recall asks “of what was expected, how much did you cover?” BLEU is precision-oriented (for translation); ROUGE is recall-oriented (for summarisation).

BLEU. For candidate c (length c) and reference r (length r , closest length), modified n -gram precision p_n clips counts to the max in any reference. With $N = 4$, $w_n = 1/N$:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right), \quad \text{BP} = \begin{cases} 1 & c > r \\ \exp(1 - r/c) & c \leq r \end{cases} \quad (8.8)$$

BP is the *brevity penalty*: short candidates that cheat on precision are penalised. Geometric mean means one zero p_n zeroes BLEU (smoothing variants fix this). Fig. 8.3 shows clipping.

ROUGE. ROUGE- N is recall over n -grams (let $c_g = \text{count}_c(g)$, $r_g = \text{count}_r(g)$):

$$\text{ROUGE-N} = \frac{\sum_{g \in \text{ref}} \min(c_g, r_g)}{\sum_{g \in \text{ref}} r_g}. \quad (8.9)$$

ROUGE-L uses longest common subsequence: with $\text{LCS}(c, r)$,

$$R_{\text{LCS}} = \frac{|\text{LCS}|}{|r|}, \quad (8.10)$$

$$P_{\text{LCS}} = \frac{|\text{LCS}|}{|c|}, \quad (8.11)$$

$$F_{\text{LCS}} = \frac{(1 + \beta^2)RP}{R + \beta^2P}. \quad (8.12)$$

LCS captures word order without fixing n . Both BLEU and ROUGE are string-overlap proxies and correlate imperfectly with human judgement.

Listing 8.1: Nucleus (top- p) sampling and sentence-level BLEU with brevity penalty.

```

1 import numpy as np
2 from collections import Counter

```

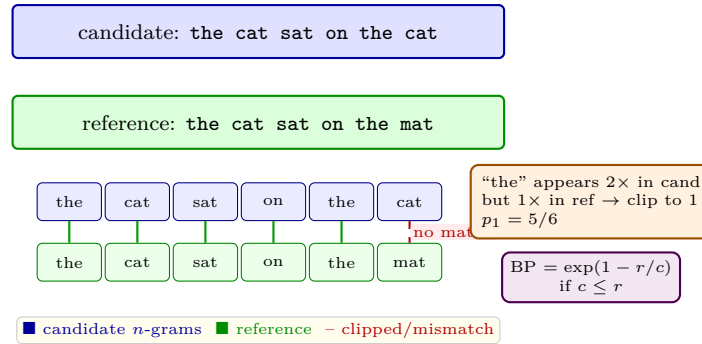


Figure 8.3: BLEU n -gram matching with clipping. Candidate “cat” matches once; the second “the” is clipped because the reference has only one “the” after “on”. Unmatched n -grams lower p_n ; short candidates are further penalised by BP.

```

3
4 def nucleus_sampling(logits, p=0.9, temperature=1.0):
5     """Sample one token from logits with temperature + top-p."""
6     logits = np.array(logits) / max(temperature, 1e-9)
7     # softmax
8     e = np.exp(logits - logits.max())
9     probs = e / e.sum()
10    # sort descending
11    idx = np.argsort(-probs)
12    sorted_p = probs[idx]
13    cumsum = np.cumsum(sorted_p)
14    # keep smallest set with cumsum >= p
15    cutoff = np.searchsorted(cumsum, p) + 1
16    keep = idx[:cutoff]
17    # renormalise
18    filt = probs[keep] / probs[keep].sum()
19    return int(np.random.choice(keep, p=filt))
20
21 # demo: peaked distribution
22 print(nucleus_sampling([3.0, 1.0, 0.2, 0.1], p=0.9, temperature=0.8))
23
24 def sentence_bleu(candidate, reference, N=4):
25     """Toy BLEU without smoothing: candidate/reference are token lists."""
26     c, r = len(candidate), len(reference)
27     BP = 1.0 if c > r else np.exp(1 - r / max(c, 1))
28     p_ns = []
29     for n in range(1, N+1):
30         cand_ngrams = Counter(tuple(candidate[i:i+n]) for i in range(c-n+1))
31         ref_ngrams = Counter(tuple(reference[i:i+n]) for i in range(r-n+1))
32         overlap = sum(min(cand_ngrams[g], ref_ngrams[g]) for g in cand_ngrams)
33         total = max(sum(cand_ngrams.values()), 1)
34         p_ns.append(overlap / total)
35     # geometric mean; 0 if any p_n == 0 (real BLEU smooths this)
36     if min(p_ns) == 0:
37         return 0.0, p_ns, BP
38     score = BP * np.exp(sum(np.log(p) for p in p_ns) / N)
39     return score, p_ns, BP
40
41 cand = "the cat sat on the cat".split()

```

```

42 | ref = "the cat sat on the mat".split()
43 | print(sentence_bleu(cand, ref, N=2)) # (BLEU, [p1,p2], BP)

```

8.5 Chapter Notes

Greedy/beam search and sampling are surveyed by Holtzman et al. (2020, nucleus sampling) and Fan et al. (2018, hierarchical stories). RLHF was scaled by Ouyang et al. (2022, InstructGPT) and Christiano et al. (2017); trust-region PPO is Schulman et al. (2017). DPO (Rafailov et al., 2023) removes the RL loop. BLEU is Papineni et al. (2002); ROUGE is Lin (2004). String overlap metrics miss semantics—learned metrics (BERTScore, COMET) and human evaluation remain essential.

8.6 Exercises

- E8.1 **Temperature arithmetic.** Logits $[2, 1, 0]$ give $P \propto [e^2, e^1, 1]$. Compute $P_{0.5}, P_1, P_2$ via Eq. (8.2). Which τ makes the top token exceed 0.9? Explain why $\tau \rightarrow 0$ recovers greedy and $\tau \rightarrow \infty$ recovers uniform.
- E8.2 **Top- k vs. top- p .** Distribution $P = [0.5, 0.3, 0.1, 0.06, 0.04]$. (a) What set does top- k with $k = 2$ keep and with what renormalised probs? (b) What set does top- p with $p = 0.85$ keep? (c) Give a peaked and a flat distribution where the two methods disagree sharply and argue which adapts better.
- E8.3 **Reward model loss.** With $r_\phi(x, y_w) = 1.2, r_\phi(x, y_l) = 0.4$, compute $\mathcal{L}_{\text{RM}}$ from Eq. (8.5) and its gradient w.r.t. $\Delta = r_w - r_l$. What happens to the gradient when $\Delta \gg 0$ vs. $\Delta \approx 0$? Relate to DPO’s implicit reward.
- E8.4 **BLEU by hand.** Candidate “the cat the cat” ($c=4$) vs. reference “the cat sat” ($r=3$). Compute clipped p_1, p_2 , BP, and BLEU-2 with $w_1 = w_2 = 0.5$. Then compute ROUGE-1 recall and explain why BLEU penalises the repetition differently than ROUGE.
- E8.5 **Build and critique.** Using the code in Listing 8.1, (a) generate 5 samples from logits $[2, 1.5, 1, 0.2]$ with $p = 0.9, \tau = 1$ vs. greedy, (b) compute BLEU-4 for candidates of length 3 vs. 10 against a length-8 reference to see BP in action, and (c) propose one failure mode of BLEU/ROUGE for dialogue and suggest a better evaluation.